

LLM Inference Scheduling: A Survey of Techniques, Frameworks, and Trade-offs

Morgan Heisler, Zahra Yousefijamarani, Xinglu Wang, Qian Wang, Ge Shi, Hanieh Sadri, Timothy Yu, Yifei Li, Haley Li, Gursimran Singh, Linzi Xing, Daesik Jang, Hong Chang, Qintao Zhang, Kan Chen, Shuming Jing, Yizhou Shan, Yi Li, Xiaolong Bai, Yu Zhang, Tianzheng Wang, Jiannan Wang, Ivan Beschastinikh, Ying Xiong, Yong Zhang, Zhenan Fan

Abstract

Large Language Models (LLMs) require substantial computational resources, making efficient inference a critical challenge. As the demand for LLMs grows in serverless and large-scale deployments, scheduling techniques play a pivotal role in optimizing inference performance, balancing latency, cost, and throughput. This survey provides a comprehensive overview of scheduling strategies for LLM serving across both the control and data planes. We explore key challenges such as cold start latency, HOL blocking, resource utilization, and dynamic workload variations. We analyze scheduling techniques, including reinforcement learning-based workload management, adaptive batching, speculative execution, and multi-tenant scheduling. Additionally, we compare scheduling frameworks across latency reduction, cost efficiency, and scalability. Finally, we identify open research directions, including ML-driven adaptive scheduling, energy-aware inference, and heterogeneous hardware utilization. This survey aims to serve as a foundational reference for researchers and practitioners optimizing LLM inference at scale and is maintained as a live document, updated monthly to reflect ongoing developments.

Keywords

autoscaling, computing and processing, inference, inference scheduling, kv cache, large language models, resource management, scheduling, serving systems, survey, system optimization

LLM Inference Scheduling: A Survey of Techniques, Frameworks, and Trade-offs

MORGAN LINDSAY HEISLER, Huawei Technologies Canada Co., Ltd., Canada
ZAHRA YOUSEFIJAMARANI, Huawei Technologies Canada Co., Ltd., Canada
XINGLU WANG, Huawei Technologies Canada Co., Ltd., Canada
QIAN WANG, Huawei Technologies Canada Co., Ltd., Canada
GE SHI, Huawei Technologies Canada Co., Ltd., Canada
HANIEH SADRI, Huawei Technologies Canada Co., Ltd., Canada
TIMOTHY YU, Huawei Technologies Canada Co., Ltd., Canada
YIFEI LI, Huawei Technologies Canada Co., Ltd., Canada
HALEY LI, Huawei Technologies Canada Co., Ltd., Canada
GURSIMRAN SINGH, Huawei Technologies Canada Co., Ltd., Canada
LINZI XING, Huawei Technologies Canada Co., Ltd., Canada
DAESIK JANG, Huawei Technologies Canada Co., Ltd., Canada
HONG CHANG, Huawei Technologies Canada Co., Ltd., Canada
QINTAO ZHANG, Huawei Technologies, Ltd., China
KAN CHEN, Huawei Technologies, Ltd., China
SHUMING JING, Huawei Technologies, Ltd., China
YIZHOU SHAN, Huawei Technologies, Ltd., China
YI LI, Huawei Technologies, Ltd., China
XIAOLONG BAI, Huawei Technologies, Ltd., China
YU ZHANG, Huawei Technologies, Ltd., China
TIANZHENG WANG, Simon Fraser University, Canada
JIANNAN WANG, Tsinghua University, China
IVAN BESCHASTNIKH, University of British Columbia, Canada
YING XIONG, Huawei Technologies Canada Co., Ltd., Canada
YONG ZHANG*, Huawei Technologies Canada Co., Ltd., Canada
ZHENAN FAN*, Huawei Technologies Canada Co., Ltd., Canada

Large Language Models (LLMs) require substantial computational resources, making efficient inference a critical challenge. As the demand for LLMs grows in serverless and large-scale deployments, scheduling techniques play a pivotal role in optimizing inference performance, balancing latency, cost, and throughput. This survey provides a comprehensive overview of scheduling strategies for LLM serving across both the control and data planes. We explore key challenges such as cold start latency, HOL blocking, resource utilization, and dynamic workload variations. We analyze scheduling techniques, including reinforcement learning-based workload management, adaptive batching, speculative execution, and multi-tenant scheduling. Additionally, we compare scheduling frameworks across latency reduction, cost efficiency, and scalability. Finally, we identify open research directions, including ML-driven adaptive scheduling, energy-aware inference, and heterogeneous hardware utilization. This survey aims to serve as a foundational reference for researchers and practitioners optimizing LLM inference at scale.

Additional Key Words and Phrases: Large Language Models, Inference Scheduling, Serving Systems, Autoscaling, Resource Management, KV Cache, System Optimization, Survey, Scheduling, Inference

*Corresponding authors: Yong Zhang (yong.zhang3@huawei.com), Zhenan Fan (zhenan.fan1@huawei.com)

1 Introduction

The rapid advancement of large language models (LLMs) has led to their widespread deployment across diverse applications, from chatbots and virtual assistants to code generation and scientific research [13]. However, the computational demands of LLM inference pose significant challenges for real-world deployment, particularly in large-scale and serverless environments. Unlike conventional deep learning models, LLMs demand substantial memory capacity, computational power, and parallelism to achieve efficient inference at scale. Importantly, the mere availability of these resources does not guarantee high performance. Their effectiveness depends on how efficiently they are orchestrated and shared across workloads. Scheduling therefore plays a pivotal role: it governs the allocation and coordination of limited resources, directly influencing latency, throughput, and cost. As the demand for LLM-powered services continues to grow, the development of robust scheduling mechanisms that can dynamically manage resources while preserving scalability, efficiency, and service-level objectives has become an essential research priority [180].

Scheduling decisions must account for unpredictable query arrival rates, model size variability, and hardware constraints, all of which impact performance [143]. Furthermore, the heterogeneous nature of deployment settings, including cloud-based inference, edge computing, and on-premise clusters, adds another layer of complexity. While cloud-based systems benefit from elastic scaling, edge environments face constraints such as limited computational resources and network variability [176]. These diverse environments require adaptable scheduling strategies to maximize resource utilization while maintaining high performance.

This survey focuses on the emerging area of LLM inference scheduling, organizing the landscape around two core architectural layers:

- The **Control Plane** (Section 5) handles global coordination, autoscaling, workload orchestration, model management, and hardware provisioning.
- The **Data Plane** (Section 6) focuses on execution-time optimization, including request routing, token-level batching, KV cache management, and runtime scheduling.

This layered perspective, illustrated in Figure 1, reflects how modern LLM inference platforms manage both infrastructure-wide scheduling and micro-level runtime efficiency by bridging system-level coordination and execution-level optimization. We also analyze real-world implementations (Section 7) and evaluate how various scheduling approaches address core LLM inference challenges (Section 8).

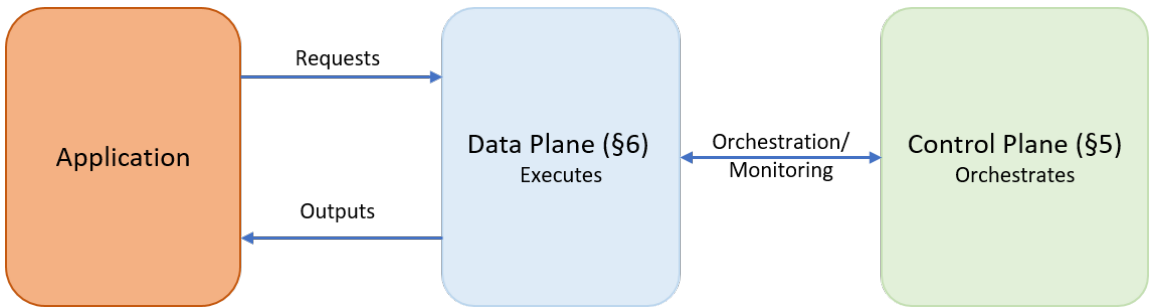


Fig. 1. High-level architecture of an LLM inference serving system, which forms the organizational basis of our survey.

While previous surveys have examined LLM inference optimizations broadly, this work focuses specifically on scheduling strategies. Prior works, such as Li et al.[84] and Miao et al.[97], have explored topics ranging from LLM

architecture to inference acceleration. However, they do not provide a deep dive into scheduling mechanisms. This survey bridges that gap by offering a focused exploration of scheduling techniques, complementing existing research and presenting new insights into how intelligent scheduling strategies can enhance LLM inference efficiency. We position our contribution relative to previous efforts in Section 4.

By synthesizing recent research and industry practices, this survey serves as a valuable resource for researchers and practitioners seeking to optimize LLM inference through better scheduling mechanisms. We identify key open challenges, such as real-time adaptability, cost optimization, and balancing trade-offs between latency and efficiency (Section 9). Addressing these challenges will drive innovation in LLM deployment, enabling more scalable and cost-effective inference solutions. Future research may explore hybrid scheduling techniques that integrate both system-level and algorithmic approaches for optimal performance. Through this work, we aim to provide a foundation for advancing the state-of-the-art in LLM scheduling.

2 Survey Methodology

To ensure both breadth and depth, our survey followed a systematic process for identifying and selecting relevant works on LLM inference scheduling and serving. We began by targeting papers published between 2020 and 2025, a period that coincides with the widespread adoption of large-scale transformer models and the emergence of system-level research on their deployment. Earlier works (e.g., TensorFlow [1], Kubernetes [16]) are included selectively to provide foundational context, but our primary emphasis is on research published from 2023 onward, when LLM serving challenges began receiving dedicated attention.

We collected candidate papers from leading venues in systems, architecture, and machine learning, including OSDI, SOSP, NSDI, EuroSys, ASPLOS, ISCA, SC, ICML, NeurIPS, and USENIX/ACM workshops. Preprints from arXiv were also considered when they introduced influential techniques or frameworks that are widely discussed or deployed. In addition, we incorporated open-source systems documentation (e.g., vLLM [171], TGI [35], Ray Serve [60], Triton [105]) where implementation details directly inform scheduling design.

Each paper was read in full by at least one co-author, with particular attention given to how scheduling, resource management, and workload orchestration were framed and evaluated. In cases where methodologies diverged across the team, we resolved differences by cross-checking against experimental details, evaluation methodology, and architectural contributions. We excluded works that only mentioned scheduling tangentially (e.g., high-level surveys on foundation models or AI infrastructure) unless they contributed specific insights into inference-time resource allocation.

The resulting corpus (180 references) reflects a balance of algorithmic proposals, systems implementations, benchmarking frameworks, and survey articles. By documenting our methodology, we aim to make our survey reproducible and extensible, providing a structured foundation that future researchers can build upon.

3 Background

Deploying LLMs at scale requires intelligent scheduling, resource allocation, and caching to handle their high computational demands, strict latency requirements, and variable workloads. As such, understanding the structure of the LLM inference pipeline and the associated bottlenecks is essential for building scalable, cost-effective, and responsive systems. This section provides an overview of the LLM inference pipeline and introduces the scheduling challenges that emerge from its architectural and operational complexity.

3.1 LLM Inference Pipeline

The inference pipeline in LLMs is a complex process that involves several key stages. Each of these stages plays a significant role in determining both the performance and efficiency of the model during inference. Understanding these components is critical for optimizing LLM serving systems and mitigating potential bottlenecks.

3.1.1 Request Processing and Model Selection. The first step in the LLM inference pipeline is **request processing**, where the incoming query is interpreted and an appropriate model is selected for the task. In a typical LLM-serving setup, model selection depends on the type of query being processed. For instance, tasks like language generation may require models like GPT[116], while classification tasks may leverage models like BERT[30]. This process involves accessing pre-trained weights or performing dynamic model swapping based on both task suitability and resource availability.

Efficient request processing and **model selection** are essential for minimizing startup latency. If not optimized, the overhead of loading models can significantly delay inference. This challenge is especially pronounced in multi-model systems or when facing high request diversity. Optimizing this stage is thus crucial for responsive pipelines.

3.1.2 Token-by-Token Generation. Following request processing, **token generation** constitutes the core computational load in most autoregressive LLMs [15, 141]. These models generate outputs incrementally (one token at a time) where each token is conditioned on previously generated tokens. While this enables context-sensitive, coherent generation, it introduces significant latency due to the repeated forward passes required for each token.

This incremental decoding structure ensures flexibility but also becomes a source of computational bottlenecks, particularly for long outputs. Optimizations like speculative decoding [19], KV caching [81, 171], and beam search [142] are often used to reduce latency, though they introduce their own engineering complexities and trade-offs.

3.1.3 Caching. To mitigate token-level latency, LLM serving systems often employ caching strategies [80, 171]. By storing intermediate results such as attention key-value (KV) pairs or past tokens, systems avoid redundant computation across forward passes or similar queries. This is particularly beneficial for interactive or repeated queries, enabling faster decoding by leveraging prior work.

Caching strategies vary in granularity and scope, from token-level KV reuse[69, 166] to full response caching[34]. However, they also introduce challenges, including cache invalidation [172], memory pressure [3], and diminishing utility[83] for highly diverse or novel queries. Caching improves latency and throughput but cannot eliminate bottlenecks from token-by-token generation alone.

3.2 Scheduling in LLM Inference

Scheduling plays a crucial role in optimizing LLM inference systems, determining how computational resources are allocated and requests are processed. Given the high computational demands of LLMs and the diversity of incoming requests, effective scheduling can significantly impact system performance. Poor scheduling can lead to inefficiencies, such as underutilization of resources or delays in response generation, which degrade user experience and increase operational costs. As such, scheduling is not just a technical requirement but a critical factor in achieving system objectives such as low latency, high throughput, fairness, and cost efficiency.

3.2.1 Key Scheduling Objectives. The primary objectives of scheduling in LLM inference revolve around four key factors: latency minimization, throughput optimization, user-level fairness, and cost efficiency. These objectives often

conflict—for example, prioritizing low-latency responses may reduce system throughput or raise operational costs. Scheduling algorithms must therefore balance these trade-offs based on system constraints, workload characteristics, and user requirements.

3.2.2 Scheduling Challenges in LLM Serving. LLM systems face several unique scheduling challenges due to their large computational footprint, heterogeneous workloads, and dynamic request patterns. These include:

Cold Start Latency and Head-of-Line (HOL) Blocking. Cold starts occur when models are loaded into memory from scratch, which is common in serverless or multi-model deployments, causing significant delay. HOL blocking arises when long-running queries delay subsequent requests in a shared queue. Both issues can severely degrade tail latency [8, 34]. Mitigations include request reordering [132], preloading [127], and model warming strategies [100].

GPU Utilization and Model Swapping. Given limited GPU memory, serving systems can swap models in and out to handle diverse workloads. Inefficient model swapping leads to fragmentation, reduced throughput, and load imbalance. Scheduling must optimize placement and timing of model loads to maintain high GPU utilization while avoiding out-of-memory errors.

Request Prioritization, Balancing SLO Guarantees and Cost. Requests often have varying latency or cost requirements, depending on user priority or application sensitivity. Scheduling must honor service-level objectives (SLOs) while minimizing the operational cost of over-provisioning. This demands dynamic prioritization, queue shaping, and admission control mechanisms that align system throughput with fairness and efficiency goals.

3.3 Evaluating LLM Scheduling Techniques

Evaluating LLM scheduling strategies requires metrics that reflect both performance and system-level efficiency.

Request Performance Metrics. End-to-end latency (e.g., P50, P95) is a key indicator of system responsiveness. Time to first token (TTFT) and time between tokens (TBT) are increasingly used for finer-grained evaluation of interactive latency [4, 40]. Throughput, measured in tokens or requests per second, reflects how well batching and scheduling scale under load [106, 178]. Complementing throughput, goodput measures the rate of successfully completed requests or useful tokens delivered per unit time, discounting failed or stalled queries [20]. Goodput is essential for capturing effective system productivity under real-world conditions with failures or retries.

Resource Utilization and Cost Metrics. GPU utilization, memory usage, energy consumption, and cost per token/request are key metrics. Efficient schedulers aim to minimize idle GPU cycles, reduce memory contention, and lower dollar and energy costs, as tracked by tools like FrugalGPT [20], Optimum-Benchmark [101], and Zeus [161].

Fairness and Service Quality Metrics. In multi-tenant or multi-priority environments, fairness is measured via Jain’s index [63] or tail spread metrics [26], while SLO adherence tracks the proportion of requests meeting latency targets. These metrics ensure that scheduling algorithms balance priorities and serve diverse users equitably.

Table 1 summarizes representative request workload datasets and tools used in the literature, which help simulate realistic traffic patterns for evaluating LLM scheduling systems. These datasets capture key scheduling signals such as arrival patterns, input/output token counts, latency, and cache behavior. It is important to note that these workloads are agnostic to the underlying LLM task and the actual inference tasks (e.g., summarization, question-answering) can vary independently. As illustrated in Figure 1, these workload traces can be used to dictate how different task types are presented to the API gateway, enabling end-to-end evaluation of scheduling strategies under realistic and diverse service conditions.

Table 1. Comparison of LLM Serving Workload Datasets by Metrics Captured

Dataset / Tool	Source & Type	Metrics / Signals Captured
BurstGPT [144]	Azure logs (real), ~121 days	Arrival rate, burstiness, input/output tokens, failures
Azure Inference Traces [111, 124]	Azure logs (real)	Input/output tokens, request latency, durations
ServeGen [153]	Alibaba Cloud (synthetic generator)	Synthetic RPS, input/output length, model ID, latency
Mooncake [114]	Kimi (Moonshot AI), ~23,000 requests	Request arrival timestamps, input/output tokens, cache hit ratios, latency metrics (TTFT, time per output token)

4 Related Surveys and Positioning

Recent surveys on LLM inference have synthesized progress across optimization techniques, serving architectures, and deployment strategies, reflecting the field’s rapid evolution. However, they vary in scope and depth, particularly regarding scheduling-related challenges. This section compares representative works and highlights how our survey offers a distinct, focused contribution.

Zhou et al.[180] cover a broad range of efficiency techniques including compression, quantization, and sparsity, with some discussion of scheduling as a subcomponent of system optimization. However, they do not distinguish control and data plane responsibilities. Similarly, Miao et al.[97] review end-to-end serving stacks but lack a structured taxonomy or focused analysis of scheduling mechanisms.

System-architecture-focused surveys such as Zhen et al.[174] and Li et al.[84] highlight decoupled serving designs and component interactions but offer only high-level treatment of scheduling, without performance metrics or coordination-layer analysis. Pan and Li[108] provide comprehensive coverage of inference systems, including request processing, model execution, and resource management. Although scheduling is included, it is not treated systematically and lacks an explicit layering or metric-oriented framework.

Other surveys address complementary but orthogonal topics. Dong et al.[32] focus on inference-time self-improvement, while Gao et al.[43] and Guo et al.[49] explore retrieval-augmented generation and multi-agent LLM systems, respectively. While relevant to runtime efficiency, they do not examine scheduling infrastructure or system-level coordination.

Table 2 summarizes key distinctions among recent surveys. Notably, no prior work treats LLM inference scheduling as a core systems challenge, and only offer varying degrees of attention to scheduling. In contrast, our survey places scheduling at the forefront, offering a taxonomy that spans both control and data plane scheduling, and organizing the space around performance-critical axes such as SLO adherence, tail latency mitigation, and multi-model coordination.

Table 2. Comparison with Related Surveys on LLM Inference

Survey	Focus	Covers Scheduling	Control/Data Plane	Recent Works (2024–25)	Metrics	Taxonomy
[97]	System-level efficiency stack	✗(Mentioned)	✗	✗	✓(Brief)	✓(Broad)
[32]	General inference efficiency	✓(Broad)	✗	✓	✗	✓(General)
[84]	Deployment/serving infrastructure	✓(Limited)	✗	✓	✗	✓
[174]	Inference serving architectures	✓(High-level)	✓(Implied)	✓	✓(Latency)	✓(Hierarchical)
[108]	Broad LLM inference systems	✓(Broad, system-level)	✗	✓	✓(Latency, cost)	✗
This Work	Inference scheduling taxonomy	✓(Detailed)	✓(Explicit)	✓	✓(SLO, fairness, tail)	✓(Layered, actionable)

While prior surveys have laid important groundwork in documenting the evolution of LLM serving systems, our contribution lies in elevating scheduling to a first-class systems challenge and organizing recent developments in a way that reveals both structural patterns and open research directions. As such, this survey complements and extends existing literature by providing a targeted, performance-aware, and future-oriented view of inference scheduling in modern LLM serving infrastructure.

5 Control Plane in LLM Serving

In large-scale LLM serving systems, the control plane functions as the policy-driven brain of the infrastructure, managing the dynamic interplay between models, compute resources, and inference workloads [44]. While the data plane executes inference through dispatching, scheduling, and runtime optimization, the control plane determines the higher-level policies and parameters that shape these scheduling decisions. In this sense, the control plane can be viewed as performing a form of meta-scheduling: it configures, constrains, and adapts the data plane’s scheduling behavior to align with performance objectives, cost constraints, and operational policies [173]. By maintaining a global, real-time view of system state, it enables centralized decision-making that balances elasticity, latency, throughput, and fairness (see Figure 2 for a control plane–focused illustration).

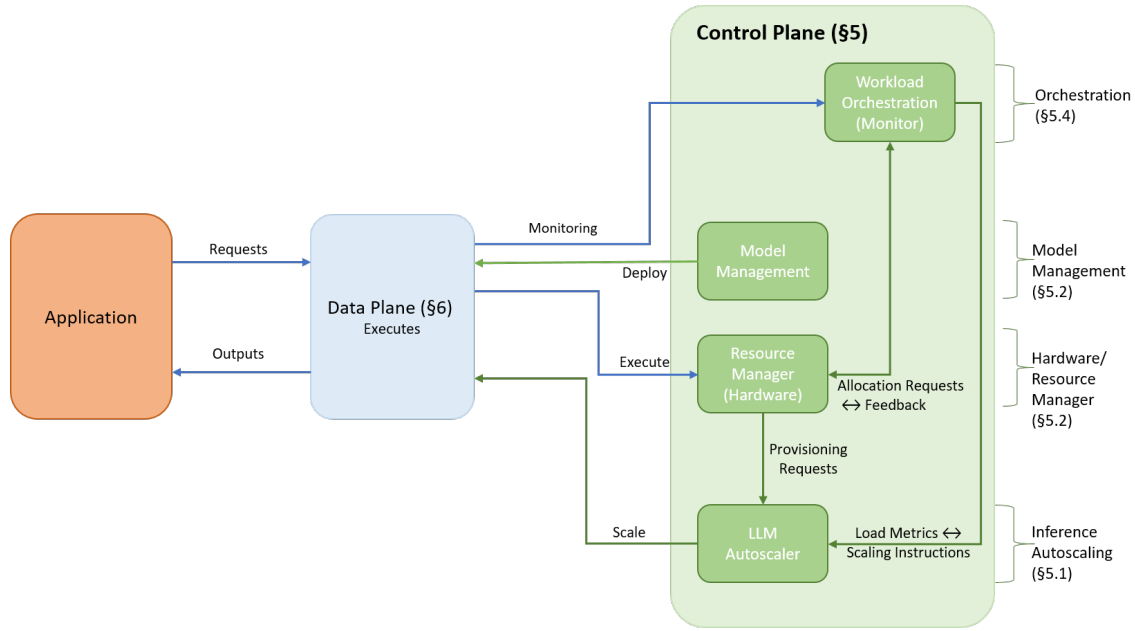


Fig. 2. Control plane–focused view of LLM serving architecture. The control plane is expanded into its key functions (workload orchestration, autoscaling, hardware resource management, and model management) while the data plane is shown in abstract form.

This section surveys the architectural components and algorithmic techniques that underpin modern control planes in LLM scheduling systems. Figure 3 outlines the structure of the section, which is organized along four key axes. Section 5.1 focuses on autoscaling, examining mechanisms for dynamically provisioning resources based on demand, with particular attention to inference autoscaling techniques that adjust capacity in response to traffic patterns. Section 5.2 addresses model and resource management, emphasizing the importance of controlling the full lifecycle of LLM artifacts, including registration, versioning, adapter deployment, and eviction. This subsection highlights strategies for managing heterogeneous models and enforcing policy constraints while minimizing cold start latencies. Section 5.3 turns to the hardware and resource manager, which is responsible for fine-grained placement and maximizing utilization efficiency under compute, memory, and network trade-offs. Finally, Section 5.4 discusses workload orchestration as a core responsibility of the control plane, detailing how it coordinates inference jobs across distributed infrastructure while ensuring consistency, responsiveness, and adherence to service-level objectives.

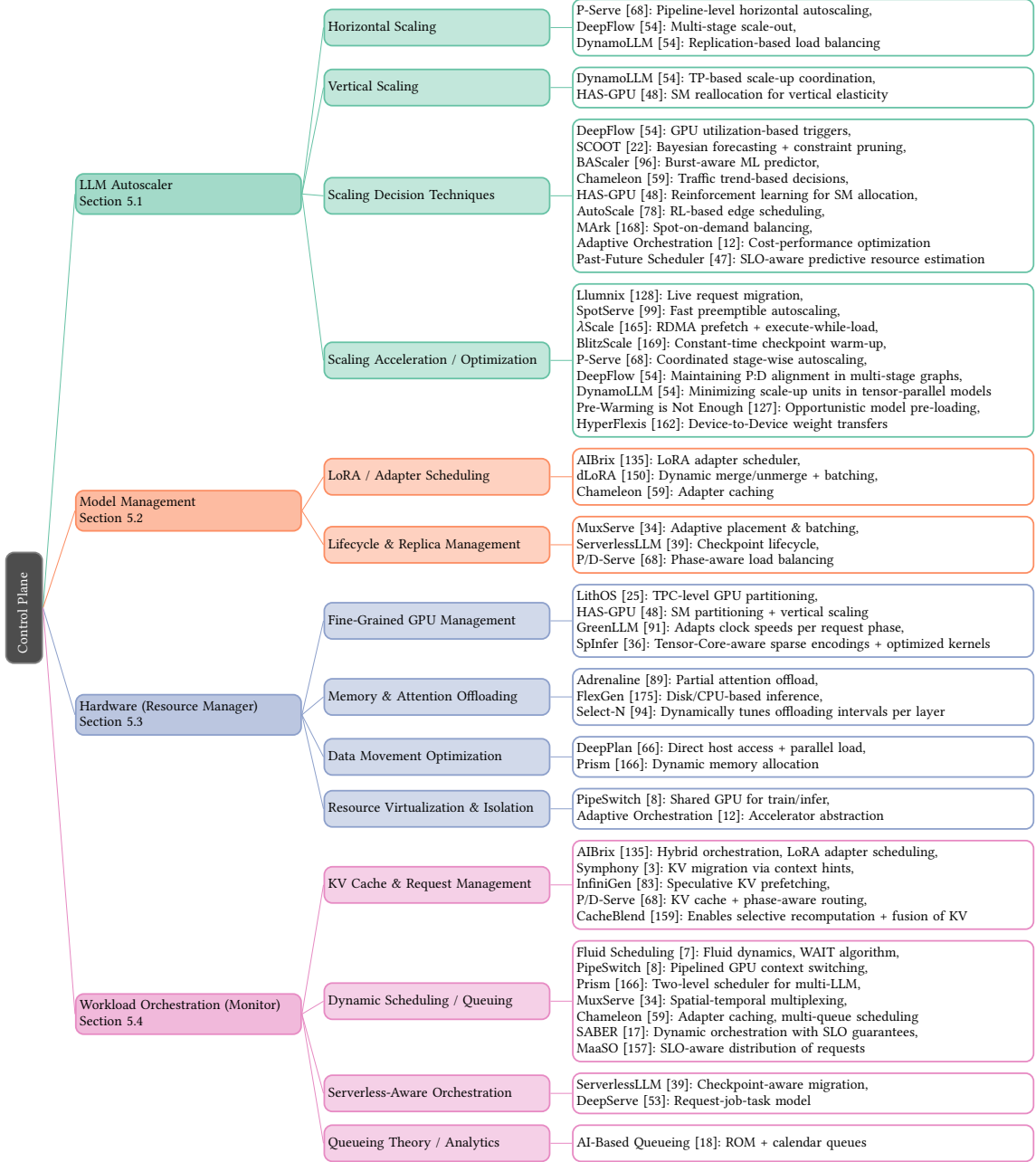


Fig. 3. Summary of Control Plane Components and Key References

Together, these components define the operational logic that enables LLM systems to scale effectively, respond adaptively, and enforce nuanced policy controls across multi-tenant environments.

5.1 Autoscaling Strategies for LLM Inference

Autoscaling is a fundamental mechanism in LLM inference systems, enabling dynamic adjustment of compute resources in response to fluctuating demand, latency constraints, and cost-performance trade-offs. Unlike conventional stateless microservices, LLM serving presents unique challenges due to its high resource requirements, coarse-grained parallelism units, and phase-separated execution pipeline. Recent research has focused on improving autoscaling responsiveness, efficiency, and adaptability across a range of deployment architectures and operational objectives.

5.1.1 Horizontal and Vertical Scaling. A key distinction in LLM autoscaling lies in the method of scaling. Horizontal autoscaling remains the dominant paradigm in production systems, favored for its simplicity, fault isolation, and compatibility with modern orchestration platforms. Systems such as P-Serve [68], DeepFlow [54], and DynamoLLM [54], dynamically replicate model-serving instances, either entire models or discrete pipeline stages such as prefill and decode, to handle increasing load. For instance, P-Serve implements stage-wise autoscaling in disaggregated inference graphs to preserve pipeline alignment while scaling. DeepFlow [54] introduces multi-stage graph replication, balancing each stage to avoid pipeline stalls, while DynamoLLM [124] manages throughput by duplicating model replicas in response to load, subject to tensor-parallel constraints. These systems capitalize on the statelessness of individual requests or components to enable rapid scaling through container orchestration or cloud-native infrastructure.

Vertical autoscaling, though less prevalent, has gained traction in specialized contexts where fine-grained scaling or hardware optimization is necessary. Rather than replicating instances, vertical autoscaling increases the resource allocation of an existing instance—either by assigning additional GPUs to expand tensor parallelism or by migrating to more capable hardware (e.g., from A100 to H100 GPUs). DynamoLLM [124] exemplifies this approach by dynamically coordinating resource allocation within multi-GPU clusters to reduce underutilization and fragmentation. HAS-GPU [48] further enables sub-GPU vertical elasticity by reallocating streaming multiprocessors (SMs) at runtime, allowing inference engines to scale up or down within a shared GPU. While vertical scaling enhances packing efficiency and can reduce orchestration overhead, it often incurs longer provisioning times due to the need for instance reconfiguration or model reloading, making it less suitable for sudden demand spikes.

5.1.2 Autoscaling Decision Techniques. Deciding when and how to scale is equally critical. Traditional threshold-based methods, as employed in DeepFlow [54] and similar systems, monitor utilization metrics such as GPU occupancy or request backlog and trigger scaling when predefined thresholds are breached. Although simple and interpretable, such reactive policies are often insufficient for particularly bursty and non-stationary workloads. Consequently, predictive autoscaling techniques have emerged, using time-series analysis or machine learning to forecast future demand. SCOOT [22], for example, combines Bayesian forecasting with constraint pruning to identify scaling actions that optimize SLOs. Similarly, BAScaler [96] incorporates burst detection to preemptively allocate resources ahead of workload surges, reducing latency violations. Past-Future Scheduler [47] advances this line of work by employing SLO-aware predictive memory footprint estimation and admission control, enabling proactive autoscaling decisions that maintain SLO guarantees under variable and unpredictable workloads.

Beyond heuristic and predictive strategies, reinforcement learning (RL) has become an increasingly popular tool for autoscaling control. RL-based systems treat autoscaling as a sequential decision-making problem, learning policies that optimize cumulative performance metrics such as latency, cost, or throughput. HAS-GPU [48] and AutoScale [78] adopt this paradigm to adaptively allocate compute resources across shared environments, while MARk [168] balances spot and on-demand instance usage under budget constraints. Chameleon [59] and Adaptive Orchestration [12] further

demonstrate the potential of workload-aware autoscaling, using dynamic models to trade off resource provisioning against cost and latency metrics in hybrid edge-cloud settings.

5.1.3 Scaling Acceleration/Optimization. An active area of research focuses on mitigating the latency overheads associated with scaling, particularly for large models with slow startup times. Cold starts, weight initialization, and GPU interconnect reconfiguration can significantly degrade user-perceived performance. Llumnix [128] addresses this by supporting live request migration, allowing workloads to be shifted between nodes without interruption. SpotServe [99] accelerates autoscaling under preemptible infrastructure by decoupling model reuse from instance lifecycle, enabling low-cost, fast-provisioning autoscale events. λ Scale [165] uses RDMA to prefetch model weights while the target GPU is still in use, thereby overlapping load time with execution. Similarly, HyperFlexis [162] reduces cold-start latency through device-to-device weight transfers, enabling rapid scaling while maintaining multi-SLO compliance. BlitzScale [169] eliminates checkpoint loading latency via a constant-time warm-up mechanism, substantially improving responsiveness to bursty traffic. Pre-Warming is Not Enough [127] builds on this theme by opportunistically pre-loading model weights based on workload heuristics, further reducing cold start delays in serverless environments.

A number of systems also focus on reducing the cost or rigidity of scaling decisions. P-Serve [68] and DeepFlow [54] maintain alignment between pipeline stages during scaling to prevent bottlenecks due to imbalanced capacity. DynamoLLM [124] minimizes the minimum viable scaling unit by coordinating smaller GPU groups across tensor-parallel configurations, enabling more flexible provisioning even under large model constraints. These approaches are particularly relevant in disaggregated inference systems, where different stages such as tokenization, model compute, and decoding may be scaled independently but must still maintain synchronized throughput.

5.1.4 Discussion and Future Outlook. Despite progress in LLM autoscaling, several key limitations remain. In practice, the responsiveness of scaling actions is constrained by provisioning delays and cold-start overheads, leading many systems to rely on conservative overprovisioning to maintain SLO compliance. Vertical scaling remains particularly disruptive, as it often entails service restarts, introducing downtime that may be unacceptable in latency-sensitive applications. Even horizontal scaling introduces transient performance degradation during reconfiguration, as system resources are temporarily rebalanced.

Granularity is another critical constraint. In tensor-parallel model deployments, the scaling unit is frequently a multi-GPU group, limiting the system’s ability to perform fine-grained adaptations and leading to inefficient resource utilization under modest load shifts. For pipeline-disaggregated systems, scaling must preserve calibrated ratios between pipeline stages, often requiring coordinated scaling actions across multiple GPUs. These composite actions introduce orchestration overhead and increase the complexity of scale planning, particularly under dynamic demand.

In heterogeneous clusters, resource fragmentation further complicates scaling. Differences in GPU memory sizes, interconnect speeds (e.g., PCIe vs NVLink), and compute capabilities can prevent available nodes from being usable for certain models or parallelism configurations. As a result, even when sufficient total resources exist, scaling decisions may be blocked by hardware incompatibility, leading to poor model placement or load imbalance. Cooldown policies further reduce agility by delaying responses to transient spikes.

Future autoscaling systems must move beyond reactive heuristics toward predictive, closed-loop strategies that integrate workload forecasting, dispatch policies, and model residency. Finer-grained disaggregation (e.g., expert or adapter-level scaling) and tighter coordination across control layers offer promising directions for improving elasticity, cost-efficiency, and SLO compliance.

5.2 Model Management

Efficient model management is essential for scalable and low-latency inference in multi-model, multi-node serving systems. As modern inference workloads often involve handling requests across diverse models and dynamically changing access patterns, systems must optimize not only which compute node executes a request, but also how models are loaded, retained, and evicted from memory. This section explores key techniques for managing models in such environments: (i) model-cache-aware scheduling, where requests are directed to nodes with models already loaded to minimize cold-start latency; (ii) demand-driven preloading, where models are proactively loaded based on predicted demand to mitigate future load imbalances; and (iii) eviction policies, where low-utilization or non-critical models are selectively offloaded to free up memory for high-priority models. Together, these strategies enable responsive, resource-efficient inference serving at scale.

5.2.1 Model-aware scheduling. In distributed and multi-node machine learning systems, minimizing model loading overhead is crucial for efficient inference and serving latency. Cache-aware scheduling in this context refers to prioritizing the dispatch of inference requests to compute nodes where the required model is already loaded in memory. This avoids redundant model loading operations and significantly reduces cold-start latencies.

When a model is not resident in a node’s memory (e.g., GPU VRAM or main RAM), the system must retrieve it from persistent storage, which can introduce substantial delays. By contrast, scheduling tasks to nodes which the model weights are already present, ensures immediate availability for execution and enables lower latency response times.

This strategy is particularly important in environments where multiple models are served concurrently, such as in multi-tenant inference clusters. By maintaining metadata about model residency across nodes, the scheduler can make better dispatch decisions, balancing load while also maximizing reuse of loaded models.

Several recent systems have adopted this strategy. Prism [166] introduces a global scheduling mechanism that determines the placement of models across GPUs based on their resource demands, aiming to balance the load and avoid bottlenecks. Inference requests for a particular model are then directed to the assigned GPUs. A GPU-local scheduler further manages requests for colocated models based on their priorities.

Similarly, ServerlessLLM [39] assesses the availability of model checkpoints across servers and estimates model loading times from different storage tiers. It dispatches inference requests to the servers that can serve them with the lowest startup latency. Another example is dLoRA [150], which supports inference serving for LoRA-fine-tuned models. Since multiple LoRA adapters may exist for a base model, dLoRA uses an adapter-aware dispatch policy, routing requests to replicas that have the appropriate adapter already loaded into memory. This minimizes the overhead of adapter loading and enables more efficient multi-model serving. AIBrix [135] also provides dynamic adapter loading and unloading, LoRA-aware routing to enhance inference efficiency.

However, while this scheduling improves latency, it can lead to severe load imbalance and increased queuing delays for requests if not properly managed.

5.2.2 Demand-driven preloading. While model-cache-aware scheduling helps reduce cold-start latency by directing requests to nodes with the required models already in memory, it does not proactively prepare for future demand. As a result, when workload patterns shift or request distributions become bursty, systems may still experience frequent model loading delays. To address this limitation, demand-driven preloading complements cache-aware scheduling by predicting future model usage and preloading models before requests arrive.

Demand-driven preloading uses historical access logs, current traffic trends, or user behavior to forecast which models are likely to be requested soon. Based on these predictions, the system selectively preloads those models onto compute nodes — ideally without disrupting current workloads or over-consuming memory. This proactive approach ensures that frequently requested models are already resident in memory when needed, reducing latency spikes during peak loads or workload shifts.

For example, dLoRA [150], in addition to its adapter-aware dispatching, incorporates demand-driven preloading by tracking adapter popularity over time. When a particular LoRA adapter shows signs of rising usage, the system preloads it to replicas before demand surges. This significantly reduces adapter load latency and smooths out request response times.

The key to effective demand-driven preloading lies in balancing resource readiness with memory constraints. Preloading too aggressively can evict currently hot models or occupy GPU memory unnecessarily, while being too conservative may fail to hide model load latency. Thus, systems often pair demand-driven preloading with intelligent eviction policies and concurrency control to maintain a high hit rate without overloading compute or memory resources.

5.2.3 Eviction Policies. As systems employ model-cache-aware scheduling and demand-driven preloading to optimize latency, they must also manage limited memory capacity on compute nodes. Without effective memory management, aggressive preloading or model retention can lead to memory exhaustion and degraded performance. This makes eviction policies a critical component of model management strategies, ensuring that less useful models are removed in a timely manner to make room for those with higher utility.

A common approach is to evict models based on recency of use, such as the Least Recently Used (LRU) policy. However, LRU alone can be suboptimal in multi-model workloads with skewed popularity or varying model sizes. Chameleon [59] addresses these limitations by proposing a cost-aware compound eviction strategy tailored for LLMs with adapter modules. Instead of relying solely on recency, Chameleon scores each adapter based on a combination of usage frequency, access recency, and memory footprint. Adapters with the lowest score, indicating lower future utility and higher eviction value, are removed first. This reduces cache miss rates and avoids unnecessary bandwidth consumption, particularly for large adapters with high loading latency.

Eviction decisions also impact the effectiveness of preloading strategies. If a preloaded model is evicted too soon due to short-term inactivity, the benefit of demand-driven preparation is lost. Hence, eviction must be carefully coordinated with preloading and scheduling. Prism [166] reflects this coordination by evicting idle models only when GPU resources are constrained and by prioritizing eviction of models with looser SLOs (service-level objectives), thereby preserving those likely to impact latency-sensitive workloads.

In summary, eviction policies serve as the backbone for managing model residency in memory-constrained environments. When integrated with cache-aware scheduling and predictive preloading, adaptive eviction strategies can significantly improve system responsiveness, maintain high memory efficiency, and reduce unnecessary data transfers in multi-model, multi-node serving systems.

5.2.4 Discussion and Future Outlook. Model management has emerged as a central challenge in modern LLM inference systems, especially as multi-model deployments become the norm. Current strategies, such as cache-aware dispatching, demand-driven preloading, and adaptive eviction, have significantly improved responsiveness by reducing cold-start overhead and increasing model availability, as mentioned before. However, each of these approaches must operate under tight resource constraints and unpredictable workload dynamics, making trade-off decisions inherently complex.

Looking forward, several promising directions remain underexplored. First, model-aware scheduling policies must increasingly consider not just memory residency but also compute contention, I/O bandwidth, and job-specific Quality-of-Service (QoS) constraints. This calls for holistic, multi-resource scheduling frameworks. Another frontier involves a growing opportunity to employ learning-based methods (e.g., reinforcement learning or bandit algorithms) to guide model retention and eviction decisions based on observed traffic and performance feedback. Ultimately, the future of efficient LLM serving hinges on developing model management policies that are not only responsive and resource-aware but also adaptive to workload evolution, system heterogeneity, and user-defined priorities. Coordinated, metric-driven model management remains an open and impactful area for systems innovation.

5.3 Hardware Resource Manager

Efficient resource management at the hardware level is foundational to scalable LLM inference. As models grow larger and more users concurrently access them, maximizing GPU utilization while maintaining isolation and performance predictability becomes increasingly difficult. The Hardware Resource Manager acts as a low-level orchestrator in the control plane, handling GPU partitioning, memory movement, and compute virtualization. Recent research introduces mechanisms that trade off isolation, granularity, and latency overhead in different ways, aiming to unlock fine-grained multi-tenancy without sacrificing tail latency or throughput.

5.3.1 Fine-Grained GPU Management. Traditional GPU schedulers assign entire devices per job, leading to underutilization and inefficient resource multiplexing. LithOS [25] challenges this by introducing TPC-level GPU partitioning, which allows multiple LLM inference kernels to run simultaneously in isolated instruction streams. Unlike CUDA streams or MIG (Multi-Instance GPU), TPCs provide finer control over execution, enabling intra-kernel isolation. In contrast, HAS-GPU [48] targets SM-level partitioning, adjusting compute allocation dynamically at runtime in response to load bursts. While LithOS emphasizes determinism and static resource partitioning, HAS-GPU supports adaptive vertical scaling, trading isolation for elasticity. The two systems reflect a broader tension between performance isolation (favoring predictable latencies) and elastic sharing (favoring high utilization).

Recent work has extended these ideas along complementary axes. GreenLLM [91] applies SLO-aware dynamic frequency scaling to further optimize GPU energy usage, adapting clock speeds per request phase (prefill vs. decode) to reduce power without violating latency targets. SpInfer [36] focuses on memory- and compute-efficient execution of sparsified LLMs, leveraging Tensor-Core-aware sparse encodings and optimized kernels to accelerate inference while minimizing memory overhead. These approaches highlight the increasing sophistication of fine-grained GPU management, encompassing not only allocation and isolation, but also dynamic energy control and sparsity-aware computation.

5.3.2 Memory and Attention Offloading. Inference workloads with long contexts face acute memory pressure due to the quadratic scaling of attention and KV cache requirements. Adrenaline [89] introduces partial attention offloading, pushing intermediate attention blocks to host memory during token generation. This reduces peak GPU memory usage while maintaining acceptable latency. FlexGen [175] extends this concept significantly by performing most of the computation off-device, using a hybrid CPU/disk scheduler to orchestrate tensor execution. Select-N [94] further advances memory offloading by explicitly considering latency SLOs, dynamically tuning offloading intervals per layer to maximize host memory usage without violating latency constraints. While Adrenaline targets modest memory savings with low overhead, FlexGen sacrifices latency to enable inference on GPUs with as little as 2GB memory. These approaches offer a spectrum of memory trade-offs: Adrenaline suits latency-critical scenarios, Select-N balances latency

and memory efficiency, whereas FlexGen enables ultra-low-cost inference on resource-constrained edge or academic environments.

5.3.3 Data Movement Optimization. Reducing the latency of memory transfers, particularly during model initialization and checkpoint loading, is another key direction. DeepPlan [66] focuses on host-device pipeline optimization, enabling direct host access to model weights and parallel data movement to reduce cold-start time. Its strength lies in batch loading of large models during system warm-up phases. Meanwhile, Prism [166] includes a dynamic GPU memory allocator that responds to runtime signals such as active context size and model residency patterns. Unlike DeepPlan, which targets load-time efficiency, Prism balances GPU memory reuse and eviction during inference. The two systems highlight temporal trade-offs: DeepPlan optimizes the system’s boot and reconfiguration time, while Prism supports continuous adaptation during live workloads.

5.3.4 Resource Virtualization and Isolation. Serving mixed workloads (e.g., concurrent training and inference jobs) requires isolating execution contexts while avoiding resource idling. PipeSwitch [8] tackles this via GPU context switching, allowing fast transitions between training and inference by retaining memory layouts in GPU-resident pools. It leverages lazy eviction and memory mirroring to minimize overhead, achieving sub-millisecond switching latencies. Adaptive Orchestration [12], a more holistic system, implements hardware abstraction layers to unify scheduling across device types and vendors. This abstraction simplifies control-plane logic and enables cross-hardware load balancing, but at the cost of losing hardware-specific tuning opportunities. PipeSwitch is highly optimized for intra-GPU multiplexing, while Adaptive Orchestration is more cross-platform and policy-driven.

5.3.5 Discussion and Future Work. Together, these innovations signal a shift toward modular, fine-tuned hardware management systems where components interact at sub-device granularity and memory is treated as a shared, dynamic resource. The next generation of LLM inference stacks will likely benefit from co-design across scheduling, memory allocation, and accelerator abstraction—especially as heterogeneous compute (e.g., CPUs, GPUs, ASICs) becomes more prevalent in serving clusters.

5.4 Orchestration

In large-scale LLM serving systems, workload orchestration in the control plane plays a vital role in maintaining system responsiveness, SLO compliance, and resource efficiency. This orchestration layer oversees request routing, batching, queueing, and system state monitoring, ensuring that workloads are dynamically scheduled and balanced according to real-time constraints such as latency budgets, GPU availability, and KV cache state.

5.4.1 Cache-aware Routing and Request Management. The first crucial responsibility of this orchestration layer is to route incoming requests to model replicas in a cache-efficient manner, which is increasingly critical as inference workloads scale. AIBrix [135] introduces a hybrid orchestration framework that tightly integrates with adapter scheduling and KV cache locality, enabling dynamic placement of requests across Ray and Kubernetes clusters. Symphony [3] enhances cache efficiency by introducing “contextual hints” to inform routing decisions that maximize KV reuse, thereby minimizing redundant decoding. InfiniGen [83] takes a speculative approach to cache management, prefetching KV entries based on predicted future access patterns. CacheBlend [159] further improves cache-aware routing in retrieval-augmented generation (RAG) workloads by enabling selective recomputation and fusion of precomputed KV caches from multiple retrieved text chunks, allowing requests to benefit from cached context even when the reused

text is not the input prefix. P/D-Serve extends these ideas with a phase-aware routing mechanism that accounts for attention-heavy versus generation-heavy stages to better align cache reuse and GPU allocation.

While this section focuses on the control plane’s responsibility in shaping policy-level decisions (such as cache-aware request routing, speculative prefetching, or phase-based scheduling) the data plane (see Section 6.4) is responsible for efficiently executing these decisions. Data plane techniques handle the low-level mechanics of KV reuse, migration, compression, and unified caching across models and hardware, ensuring the physical cache layout and memory transfers align with the high-level orchestration goals.

5.4.2 Dynamic Scheduling and Queueing. The first crucial responsibility of this orchestration layer is to route incoming requests to model replicas in a cache-efficient manner, which is increasingly critical as inference workloads scale. Fluid Scheduling [7] models request dynamics using a fluid approximation, introducing the WAIT algorithm to rebalance latency-sensitive versus throughput-bound jobs. PipeSwitch [8] enables efficient context switching between different LLM pipelines by reusing GPU memory layouts, while Prism employs a two-level scheduling mechanism to co-locate multiple LLMs with distinct service profiles. MuxServe [34] presents a spatiotemporal multiplexing strategy that interleaves requests across models and devices based on real-time usage metrics. Complementarily, Chameleon [59] applies adapter-level caching and multi-queue scheduling to enable prompt reuse and reduce load imbalance across accelerators.

Beyond throughput- and cache-oriented strategies, recent control-plane schedulers also incorporate SLO awareness. SABER [17] integrates offline latency modeling with online adaptive batching, dynamically adjusting batching policies to minimize deadline misses without sacrificing throughput. Similarly, MaaSO [157] leverages heterogeneity across model instances by profiling performance, optimizing placement, and distributing requests in an SLO-aware manner, improving both latency and overall service efficiency. While this section emphasizes control-plane dynamic scheduling, additional SLO-aware mechanisms operate in the data plane (Section 6.2.3), including runtime-level batching, KV cache management, and instance-level optimizations that further reduce latency and improve deadline adherence.

5.4.3 Serverless-Aware Orchestration. Complementing the above paradigms, recent systems extend orchestration mechanisms to serverless LLM inference settings, where challenges like cold-start latency and stateless execution complicate coordination. ServerlessLLM [38] incorporates checkpoint-aware migration strategies that minimize cold-start penalties during request redirection. On the other hand, DeepServe [53] formalizes orchestration into a three-tier model of request-job-task mappings, allowing hierarchical scheduling, and checkpoint propagation in stateless environments.

5.4.4 Queueing Theory. Beyond system-specific designs, theoretical underpinnings from queueing theory continue to inform LLM workload management. Recent efforts explore AI-enhanced scheduling policies, such as ROM-based scheduling and calendar queues [18], to predict and prioritize request flows in multi-tenant environments. These analytics-driven methods can complement existing dynamic batchers by improving fairness and anticipating burst traffic more accurately.

5.4.5 Discussion and Future Work. Together, these works reflect a broad trend: workload orchestration is evolving from static routing and batching toward adaptive, cache-aware, and latency-sensitive coordination across distributed infrastructure. Future research in this space may benefit from tighter integration with autoscaling, failure remediation, and request forecasting mechanisms, enabling orchestration systems to not only react to current workloads, but

to anticipate and optimize for future demand. Such orchestration frameworks will be critical for achieving robust, cost-efficient, and SLO-compliant LLM serving at scale.

6 Data Plane Scheduling Strategies for LLM Serving

The data plane is the critical execution layer responsible for handling inference requests and realizing the performance objectives set by the control plane. While the control plane oversees global orchestration, resource management, and policy enforcement across clusters and regions, the data plane operates closer to the hardware, focusing on scheduling, dispatching, and executing inference workloads within individual clusters or compute nodes (see Figure 4 for a data plane-focused illustration). Its design directly impacts throughput, latency, and hardware utilization, all essential to meeting strict SLOs and user experience expectations.

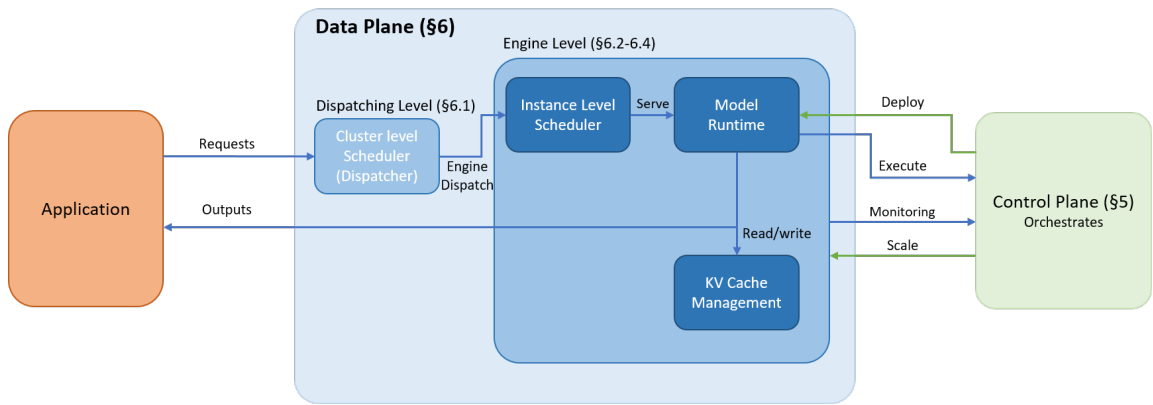


Fig. 4. Data plane-focused view of LLM serving architecture. The data plane is expanded into its primary components (cluster-level scheduling, instance-level scheduling, model runtime, and KV cache management) while the control plane is shown in abstract form.

Efficiently scheduling inference requests across available compute backends under dynamic and unpredictable workloads is a core challenge at the data plane level. This includes load balancing across model replicas, minimizing tail latency, and maximizing locality by routing requests to warm model instances or cached states. The data plane must tightly integrate with control-plane policies, enabling real-time, low-latency decision-making to adapt to fluctuating demand and resource availability.

This section systematically surveys the data plane components involved in LLM serving scheduling, organized into four main sections as shown in Figure 5: Cluster-Level Scheduler (Section 6.1), Instance-Level Scheduler (Section 6.2), Model Runtime (Section 6.3), KV Cache Management (Section 6.4). By exploring these sections, we highlight state-of-the-art mechanisms for load balancing, fine-grained scheduling, and hardware-aware execution that enable scalable, high-performance LLM inference in production environments.

6.1 Cluster-Level Dispatching Strategies

At the front of distributed LLM serving systems, cluster-level schedulers are responsible for routing incoming inference requests across a pool of compute nodes or model-serving instances. Their objective is to optimize both system efficiency, such as GPU utilization and global throughput, and user-facing performance like TTFT and end-to-end latency.

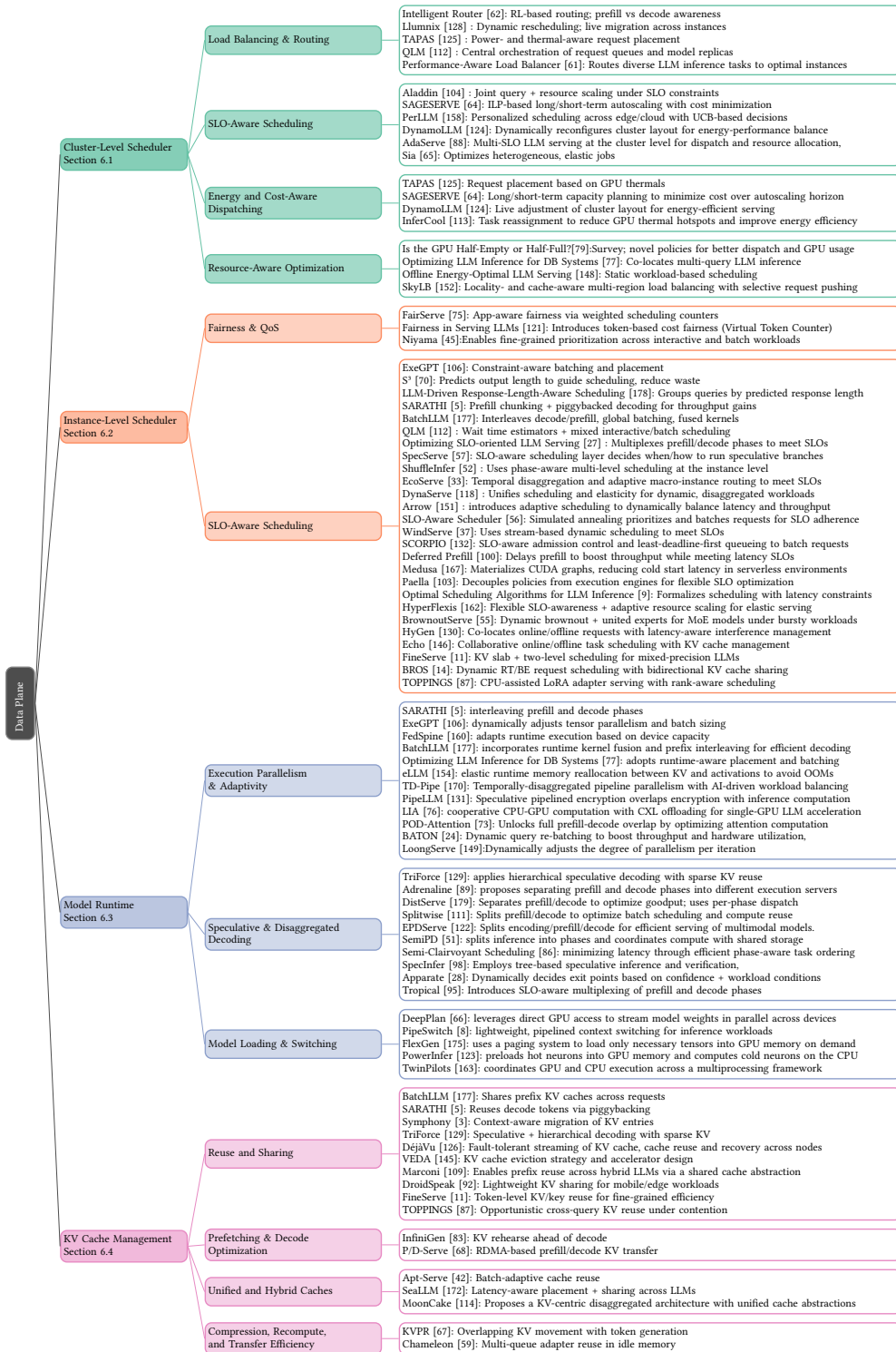


Fig. 5. Summary of Data Plane Components and Key References

Dispatching Scope and Design Dimensions Schedulers must determine not only where to place a request but also when to do so. This requires balancing node-local queue states, memory usage (e.g., KV-cache availability), and request-specific requirements such as sequence length or latency target. The decision space may span centralized queues (e.g., QLM), per-instance routing (e.g., SAL), and even dynamic migration between replicas (e.g., Llumnix).

The complexity of these schedulers varies along several dimensions. First, the granularity of operation differs, with some schedulers functioning on a per-request basis while others process batches of requests. Second, their level of information awareness ranges from stateless approaches, such as round-robin scheduling, to fully informed strategies that leverage joint system knowledge, exemplified by frameworks like LLM Router and Aladdin. Finally, the underlying algorithmic designs span a spectrum from simple heuristics to more sophisticated methods involving programmatic optimization or reinforcement learning.

6.1.1 Per-Request Load Balancing and Routing. Several schedulers focus on lightweight, responsive routing based on real-time telemetry or runtime policies. Llumnix [128] employs a two-level scheduler with per-node “llumlet” agents that support live KV cache migration, enabling rescheduling of in-progress queries to reduce tail latency and defragment memory. SAL [79] operates as a polling-based analytic router that scores nodes based on token queue length and memory availability, requiring no learned models. When paired with LARRY as a local memory-aware scheduler, SAL improves TTFT with minimal orchestration overhead. In a similar vein, the Performance-Aware Load Balancer [61] profiles instance performance under mixed workloads to dynamically route requests, reducing contention and improving utilization without deep learning models. LLM Router [62] formulates the dispatch problem as a Markov Decision Process and applies reinforcement learning to learn optimal routing policies. It adapts to workload variation using predicted sequence lengths, instance load, and interference signals.

6.1.2 SLO-Aware Global Optimization. To manage system-wide constraints and long-horizon objectives, several works adopt centralized queueing with global optimization. QLM [112] maintains a global request queue and formulates dispatch as a linear program over estimated waiting times and declared SLOs. It enables queue reordering, preemption, and model switching to optimize throughput and latency jointly. Aladdin [104] addresses joint placement and autoscaling. It models latency using prefill/decode estimators and solves a bin-packing problem to find the minimum-cost configuration that satisfies all active SLOs. Aladdin reports up to 71% GPU cost savings while maintaining latency guarantees. AdaServe [88] targets multi-SLO LLM serving, optimizing at the cluster level for request dispatch and resource allocation to meet diverse latency constraints. It introduces SLO-customized speculative decoding strategies to accelerate both throughput-oriented and latency-critical requests, improving performance under mixed-service workloads. Sia [65] complements these approaches by optimizing heterogeneous, elastic jobs across GPU types and cluster loads, reducing job completion time by up to 93% and GPU usage by up to 55%, while scaling to 2000+ GPU clusters efficiently.

6.1.3 Energy and Cost-Aware Dispatching. As serving costs and energy constraints become more salient, several systems optimize cluster layout and routing under these constraints. TAPAS [125] applies a thermal- and power-aware placement strategy, dispatching requests to avoid triggering GPU throttling events. SAGESERVE [64] formulates autoscaling as an integer linear program that balances cost and load over both short-term and long-term planning horizons. DynamoLLM [124] dynamically reconfigures cluster composition in response to workload drift, balancing energy use and inference performance. PerLLM [158] applies a personalized scheduling approach using a multi-armed bandit algorithm to

dispatch queries across edge and cloud resources, learning optimal placements per user or workload type. InferCool [113] employs dynamic task reassignment to reduce GPU thermal hotspots and improve energy efficiency.

6.1.4 Resource-Aware Scheduling. Resource-aware scheduling strategies aim to optimize LLM inference by accounting for hardware utilization, workload characteristics, and deployment constraints. At the most granular level, “Is the GPU Half-Empty or Half-Full?” [79] surveys dispatching strategies and proposes load-aware policies that dynamically balance jobs across GPUs to improve utilization within a single cluster. Expanding to domain-specific workloads, Optimizing LLM Inference for DB Systems [77] explores latency-aware model reuse and co-location strategies for multi-query inference, tailoring dispatch to both query patterns and GPU sharing requirements.

Other approaches adopt a workload-centric perspective. Offline Energy-Optimal LLM Serving [148] models energy and runtime costs as functions of input and output token counts, enabling static assignment of jobs to CPU or GPU backends for energy-efficient execution in heterogeneous clusters. Extending resource-awareness to geographically distributed deployments, SkyLB [152] introduces a locality-aware multi-region load balancer for LLM inference. By aggregating diurnal traffic patterns across regions, preserving KV-cache locality, and selectively pushing requests, SkyLB balances load while allowing providers to reserve instances based on global demand rather than regional peaks.

Together, these studies illustrate a spectrum of resource-aware scheduling techniques—from GPU-level load balancing to workload- and energy-aware assignment, and finally to multi-region, cache- and traffic-aware dispatch. This progression highlights how effective scheduling increasingly integrates knowledge of both system resources and workload characteristics to optimize LLM serving efficiency.

6.1.5 Discussion and Outlook. Cluster-level schedulers span a wide design spectrum—from heuristic-based and stateless systems to complex learned or optimized strategies. Centralized solutions (e.g., QLM, Aladdin) enable holistic optimization but incur coordination overhead and centralized bottlenecks. In contrast, systems like Llumnix and SAL+LARRY embrace decentralization, telemetry-driven heuristics, and live migration for fast and lightweight adaptation.

Learned approaches (e.g., LLM Router) show strong adaptability across deployments and workload shifts but may require significant training data or tuning. Future directions could involve hybrid schedulers that blend global optimization with local responsiveness, and more cost-aware or energy-constrained dispatching models as LLM workloads scale across heterogeneous environments.

6.2 Instance-Level Scheduling Strategies

After requests are dispatched to a compute node, the instance-level scheduler is responsible for managing their execution on the local model replica. This stage governs batching, ordering, and issuing of prefill and decode operations, making it a critical bottleneck in high-throughput, low-latency serving. Instance schedulers operate under tight GPU memory constraints and must adapt to prompt heterogeneity, queue dynamics, and system-level service objectives.

6.2.1 Responsibilities and Challenges. Key responsibilities include (i) forming efficient batches that balance latency and compute utilization, (ii) maintaining queue fairness while avoiding HOL blocking, and (iii) issuing decode operations under memory and concurrency limits. The main challenges include handling variable-length inputs, managing limited KV-cache capacity, and dealing with dynamic workloads that fluctuate between interactive and bulk queries.

6.2.2 Fairness and Quality of Service. Workload fairness and responsiveness are critical in multi-tenant LLM deployments. FairServe [75] enforces application-level fairness using weighted scheduling counters, ensuring that interactive or high-priority applications are not starved by bulk workloads. Fairness in Serving LLMs [121] extends this idea using

a token-based virtual cost model, accounting for sequence length and computational load to allocate tokens more equitably across users. Niyama [45] further explores QoS-driven scheduling by enabling fine-grained prioritization across interactive and batch workloads, improving resource utilization while maintaining latency guarantees.

6.2.3 SLO-Aware Scheduling. Efficient LLM inference requires scheduling strategies that respect latency and throughput constraints while maximizing hardware utilization. Early SLO-aware approaches relied on heuristics and prediction-based techniques. For instance, S^3 [70] pioneered response-length prediction to group compatible sequences, reducing wasted KV cache memory. SARATHI [5] fused prefill and decode phases via prompt chunking and piggybacking, improving GPU pipeline utilization, while BatchLLM [177] exploited shared prefixes for batch-aware kernel fusion, further reducing decoding overhead.

Building on these foundations, phase-disaggregated and adaptive execution schedulers emerged to handle dynamic workloads. ExeGPT [106] learns execution configurations sensitive to input distributions and latency targets, and SCOOT [22] applies Bayesian optimization to fine-tune engines for improved SLO compliance. Deferred Prefill [100] selectively postpones prefill phases to maximize throughput without violating latency objectives, and Medusa [167] accelerates serverless inference by materializing CUDA graphs to reduce cold start latency. LARRY [79] prioritizes low-latency “fast-path” requests under memory pressure, highlighting the trade-offs between complexity and responsiveness. PD-Multiplexing [27] explicitly manages concurrent prefill and decode execution within an instance, while SpecServe [57] introduces adaptive speculative decoding tailored to diverse SLO profiles.

To address multi-level, elastic, and cluster-aware scheduling, several frameworks focus on resource isolation, load balancing, and SLO-driven orchestration. ShuffleInfer [52] balances prefill and decode phases across mixed workloads to reduce decode hotspots. EcoServe [33] implements proactive orchestration spanning intra- and inter-instance phases for cost-effective SLO compliance, and HyperFlexis [162] jointly optimizes scheduling and adaptive resource scaling under bursty demand. DynaServe [118] fragments requests into micro-requests dynamically scheduled across disaggregated resources, while Arrow [151] provides instance-level scheduling policies responsive to real-time resource availability. The SLO-Aware Scheduler [56] uses simulated annealing to prioritize and batch requests based on heterogeneity and runtime behavior, and WindServe [37] leverages stream-based dynamic scheduling to reduce interference between prefill and decode phases. Paella [103] decouples scheduling policies from execution engines, and Optimal Scheduling Algorithms for LLM Inference [9] formalizes latency-constrained scheduling with approximation algorithms that provide theoretical guarantees.

Recent systems also tackle hybrid, interference-aware, and precision-adaptive workloads. BrownoutServe [55] optimizes Mixture-of-Experts (MoE) models under bursty traffic, using “united experts” and a dynamic brownout mechanism to reduce inference latency while maintaining SLO compliance. HyGen [130] and Echo [146] efficiently co-locate latency-sensitive online requests with throughput-oriented offline tasks, leveraging latency prediction, KV cache management, and execution estimation to maximize overall performance. FineServe [11] supports mixed-precision models through a KV slab allocator and two-level scheduling, improving GPU memory efficiency and throughput. BROS [14] dynamically schedules hybrid real-time and best-effort requests, enabling bidirectional KV cache sharing to balance latency and throughput. Finally, TOPPINGS [87] efficiently serves LoRA adapters using CPU-assisted prefill computation and rank-aware scheduling, achieving high SLO attainment across heterogeneous requests.

Together, these works illustrate a trajectory from heuristic and prediction-based schedulers toward sophisticated, multi-phase, hybrid, and resource-adaptive SLO-aware frameworks. They highlight the importance of dynamic, fine-grained orchestration mechanisms in meeting increasingly diverse and stringent service-level objectives in modern LLM inference workloads.

6.2.4 Discussion and Future Work. Instance-level schedulers vary in the sophistication of their decision logic, from heuristics (e.g., LARRY) to predictive models (e.g., ExeGPT). Systems such as SARATHI and BatchLLM emphasize token-phase overlap and memory-aware fusion, while others (e.g., SCOOT) optimize execution knobs for heterogeneous query mixes. As serving environments scale, hybrid schedulers that integrate SLO-awareness, fairness, and cache efficiency may offer the best balance of robustness and performance.

6.3 Model Runtime

The model runtime layer executes the core inference loop, handling model loading, execution parallelism, and token generation across GPUs. As LLMs grow in size and sequence length, runtime efficiency becomes a critical bottleneck. Runtime systems must therefore coordinate tightly with memory and scheduling components to manage resources, minimize stalls, and maintain latency targets in heterogeneous serving environments.

6.3.1 Parallelism and Execution Adaptivity. A central concern in model runtime systems is maximizing hardware utilization through optimized parallelism strategies. SARATHI [5] improves pipelined decoding efficiency by interleaving prefill and decode phases, thereby reducing pipeline stalls and improving throughput. Similarly, ExeGPT [106] dynamically adjusts tensor parallelism and batch sizing based on observed sequence distributions, achieving better latency-performance trade-offs. LoongServe [149] adopts elastic sequence parallelism (ESP), dynamically adjusting the degree of parallelism per iteration based on workload and phase (prefill vs decoding). It also overlaps partial-decoding communication with computation and leverages a unified distributed KV cache to reduce fragmentation and migration overheads. BATON [24] enhances batch-wise inference efficiency by dynamically re-batching queries during runtime, improving utilization and throughput for large language models. BatchLLM [177] incorporates runtime kernel fusion and prefix interleaving to enable efficient decoding across similar-input queries. POD-Attention [73] further unlocks full overlap of prefill and decode phases by optimizing the attention computation itself, enabling faster inference through improved phase concurrency.

To address memory bottlenecks during inference, eLLM [154] introduces an elastic memory management framework that dynamically reallocates GPU memory between KV cache and intermediate activations, ensuring high memory utilization and preventing out-of-memory failures. TD-Pipe [170] proposes a temporally-disaggregated pipeline parallelism architecture that separates prefill and decode phases to reduce pipeline bubbles and improve throughput on multi-GPU systems with limited interconnect bandwidth. Its hierarchical controller and AI-driven workload balancing further optimize execution parallelism.

In federated and constrained environments, FedSpine [160] adapts runtime execution based on device capacity, dynamically switching between local and split execution paths to preserve performance on weak edge clients. Systems like Optimizing LLM Inference for DB Systems [77] adopt runtime-aware placement and batching techniques tailored to structured data workflows.

Complementing these, PipeLLM [131] introduces speculative pipelined encryption that overlaps cryptographic operations with model execution, minimizing security overhead while maintaining low-latency, high-throughput inference. LIA [76] accelerates single-GPU LLM inference by leveraging cooperative AMX-enabled CPU-GPU computation combined with CXL offloading, enhancing throughput and reducing latency through heterogeneous resource utilization.

6.3.2 Speculative and Disaggregated Execution Models. Recent advances in LLM serving architectures have increasingly embraced disaggregation and speculative execution to overcome decoding bottlenecks and improve resource utilization. Adrenaline [89] introduces a design that physically separates the prefill and decode phases onto distinct execution servers, enabling more effective memory reuse and parallelism across concurrent requests. TriForce [129] complements this by employing hierarchical speculative decoding combined with sparse KV reuse, accelerating inference especially for long-context sequences.

Building on these principles, DistServe [179] improves goodput by disaggregating the prefill and decode stages of execution across distinct hardware resources. Splitwise [111] advances phase-splitting strategies by carefully partitioning generative inference into decoupled segments, facilitating efficient hardware utilization while maintaining low latency. More recent systems, such as EPDServe [122] and ModServe [115], extend phase disaggregation to the encoding stage in large multimodal models (LMMs). EPDServe [122], in particular, introduces phase-aware parallelism, batching, scheduling, and resource management strategies—including dynamic allocation and migration—to enhance overall serving efficiency for LMMs. Extending beyond phase disaggregation, semi-PD [51] integrates phase-wise computation with a unified storage layer, enabling seamless coordination between prefill and decode operations. This holistic design reduces data movement overhead and improves fault tolerance, emphasizing the benefits of unified management in phase-disaggregated serving.

While these approaches focus primarily on phase separation and resource efficiency, Tropical [95] targets SLO attainment in disaggregated LLM serving. Tropical introduces an SLO-aware multiplexing strategy that balances queuing delays and interference between prefill and decode phases.

Speculative execution techniques further enhance these architectures by dynamically predicting and managing multiple potential decoding paths. Semi-Clairvoyant Scheduling [86] leverages partial foresight of request characteristics to schedule speculative decoding requests effectively, reducing overall inference latency by dynamically ordering phase-execution tasks based on predicted decoding costs. Complementing this, SpecInfer [98] introduces a tree-based speculative inference and verification framework that concurrently predicts multiple decoding branches and prunes unlikely paths early, accelerating inference while preserving accuracy. This rigorous verification-based speculation improves resource efficiency and latency beyond traditional speculative models. Beyond speculative decoding of output tokens, Apparate [28] rethinks early-exit strategies as a form of speculative execution for ML serving. By dynamically deciding exit points based on confidence and workload conditions, Apparate mitigates latency-throughput tensions, improving responsiveness for short requests while preserving efficiency on long sequences. This positions early-exit control as a complementary speculative mechanism alongside branch-pruning and multi-path decoding.

Together, these works reflect a broader shift towards decomposing LLM inference into modular, speculatively executed phases, enabling better parallelism, memory reuse, and adaptive scheduling. This paradigm is crucial for addressing the scaling challenges posed by growing model sizes and latency-sensitive applications.

6.3.3 Model Loading and Context Switching. Runtime efficiency also depends on the cost of model loading and job switching. DeepPlan [66] leverages direct GPU access to stream model weights in parallel across devices, significantly reducing model load latency in multi-GPU setups. PipeSwitch [8] introduces lightweight, pipelined context switching

for inference workloads, allowing models to time-share GPU resources with minimal overhead. Recent systems like FlexGen [175] and PowerInfer [123] further reduce model loading overhead by selectively transferring only active parameters into GPU memory on demand, leveraging hybrid CPU-GPU memory hierarchies. TwinPilots [163] builds on this principle by coordinating GPU and CPU execution across an asymmetric multiprocessing framework, enabling hierarchical memory management and dynamic context switching with minimal latency.

6.3.4 Discussion and Outlook. Recent runtime systems demonstrate a shift from static execution pipelines to adaptive, modular runtimes that respond to workload variation, memory availability, and system heterogeneity. Future directions include integrating runtime schedulers with cache managers and autoscalers, optimizing across the full inference lifecycle.

6.4 KV Cache Management Strategies

Transformer-based LLMs rely on KV caches to store intermediate hidden states (keys and values) during autoregressive decoding. These caches are reused across token generation steps to avoid redundant computation, making KV management a critical factor in achieving low-latency, high-throughput inference. However, the memory footprint of KV caches scales with prompt length, batch size, and number of concurrent requests, leading to memory pressure, cache thrashing, or degraded quality-of-service in real-world deployments.

6.4.1 Core Responsibilities and Challenges. KV cache management encompasses three key responsibilities: (i) reuse of previously computed KV states across tokens or queries, (ii) eviction under memory constraints with minimal recomputation penalty, and (iii) transport and sharing strategies to enable efficient cache migration across devices or systems. Several challenges persist: naïve eviction policies (e.g., LRU/FIFO) often lead to premature deletions; multi-turn or long-context scenarios exacerbate memory pressure; and sharing KV entries across requests or models remains non-trivial due to alignment, security, or latency constraints.

6.4.2 Reuse and Sharing Mechanisms. A central thread in recent work is improving reuse granularity to reduce redundant computation and memory overhead during LLM inference. BatchLLM [177] achieves prefix amortization by sharing KV caches across requests with similar input structures, while SARATHI [5] further minimizes recomputation by piggybacking on previously decoded tokens. Symphony [3] extends these ideas to multi-turn dialogue workloads, using interaction-aware context hints to proactively migrate KV entries off the critical path, thereby increasing concurrency without hurting user-perceived latency.

Beyond dialogue-level reuse, Marconi [109] demonstrates that KV sharing can extend across heterogeneous models by introducing a shared cache abstraction, while TriForce [129] combines sparse KV reuse with hierarchical layouts and speculative decoding to optimize generation for long-context sequences. DéjàVu [126] addresses scalability in disaggregated deployments by streaming KV entries across networked components for fault-tolerant, low-latency reuse. In parallel, VEDA [145] adapts reuse granularity to workload characteristics through fine-grained partitioning and eviction policies, balancing memory efficiency with inference throughput.

Building on this trajectory, recent work has begun to push reuse mechanisms toward finer granularity and more diverse deployment environments. FineServe [11] introduces token-level reuse within decoding steps, complementing prefix-based schemes with intra-sequence sharing that reduces redundant key computations. TOPPINGS [87] opportunistically coordinates reuse across concurrent requests, exploiting overlap under contention to improve throughput without degrading latency guarantees. DroidSpeak [92] extends these ideas to constrained mobile and edge settings,

showing that lightweight KV sharing can reduce on-device recomputation while operating under tight memory budgets. Together, these efforts highlight a spectrum of reuse mechanisms—from prefix-level amortization to opportunistic and fine-grained strategies—that improve efficiency across increasingly heterogeneous and resource-constrained LLM serving environments.

6.4.3 Prefetching, Phase-Aware Strategies, and Transport Optimization. To reduce latency under cache-miss-prone workloads, InfiniGen [83] introduces speculative KV prefetching, which precomputes likely decode paths in bursty workloads, improving throughput by minimizing rehearsal penalties. Meanwhile, P/D-Serve [68] decomposes KV cache access into prefill and decode phases and employs RDMA-optimized transport to reduce device-to-device migration latency, enhancing pipeline utilization in multi-node inference setups.

6.4.4 Unified and Hybrid Cache Designs. Recent systems explore hybrid caching architectures that balance storage overhead against computational savings to optimize LLM serving. Apt-Serve [42] exemplifies this by combining KV and hidden state reuse, employing batch-aware optimizers and adaptive granularity selection to maximize throughput while avoiding out-of-memory errors. Similarly, SeaLLM [172] introduces a unified cache design spanning multiple colocated LLMs with variable sequence lengths, leveraging latency-aware eviction policies and model lifecycle heuristics to efficiently manage KV reuse in multi-tenant deployments.

Complementing these approaches, Mooncake [114] proposes a KV-cache-centric disaggregated architecture specifically optimized for chatbot workloads. By trading additional storage capacity for reduced computation, Mooncake emphasizes persistent and shared KV cache structures across disaggregated components, improving latency and fault tolerance in conversational LLM serving scenarios. This KV-focused design highlights the evolving trade-offs in cache management between memory consumption and compute efficiency in large-scale LLM systems.

6.4.5 Compression, Recompute, and Transfer Efficiency. To cope with PCIe and memory transfer bottlenecks, KVPR [67] reduces cache size via partial KV recomputation and overlaps memory movement with token generation. This trade-off between compute and memory usage sustains low-latency inference under GPU resource constraints. Similarly, Chameleon [59] addresses adapter-heavy LLM settings by caching frequently used adapters and implementing a multi-queue scheduler to reduce HOL blocking.

6.4.6 Discussion and Open Directions. Recent approaches illustrate a shift from static cache policies to dynamic, phase-specific, and workload-adaptive strategies. Moving forward, KV management strategies must incorporate context-aware reuse, distributed cache coordination, and compression-aware eviction to support long-context, multi-modal, and multi-tenant LLM serving environments.

7 Case Studies and Real-World Implementations

7.1 Open-Source LLM Serving Frameworks

The rapid development and deployment of LLMs have necessitated the emergence of efficient inference frameworks that optimize memory usage, reduce latency, and maximize throughput. While proprietary solutions dominate many commercial applications, open-source frameworks have become vital tools for democratizing access to LLM capabilities. These systems offer customizable, scalable, and cost-effective alternatives, enabling researchers and developers to serve high-performance models without heavy infrastructure investment. This section highlights major open-source LLM inference frameworks and the scheduling strategies they use to improve serving efficiency.

vLLM [81] is one of the most widely adopted LLM serving frameworks, featuring the innovative paged attention mechanism and token-level scheduling for high-throughput generation with large context windows.

Aibrix [135], developed within the vLLM ecosystem, extends these capabilities to multi-tenant scenarios by introducing fine-grained scheduling policies for concurrent multi-model and multi-user serving.

Hugging Face’s Text Generation Inference (TGI) [35] provides efficient model deployment using dynamic batching and CUDA-optimized quantization.

DeepSpeed-Inference [117], developed by Microsoft, supports model parallelism and tensor slicing to scale LLM inference across multiple GPUs.

Llumnix [128] is optimized for real-time applications such as chatbots, combining load-aware batching with adaptive caching to support low-latency responses.

Beyond LLM-specific frameworks, general-purpose model serving systems such as **Ray Serve** [60], **Triton Inference Server** [105], and **TorchServe** [134] offer extensible serving infrastructures that support custom batching logic, autoscaling, and multi-model routing. While not designed exclusively for LLMs, they are often used to orchestrate inference in production deployments and integrate well with LLM backends like vLLM or TGI.

Together, these frameworks demonstrate a diverse set of design choices, from kernel-level optimizations to request-level scheduling that enable efficient and flexible LLM serving. A summary of their scheduling and execution capabilities are shown in Table 3. Notably, it reveals that although leading open-source inference frameworks differ in execution features, their scheduling policies remain relatively narrow. First-in-first-out (FIFO) remains the dominant default strategy, with only a few frameworks (e.g., Aibrix and Llumnix) exposing fairness-oriented or workload-aware extensions. This limited diversity stands in contrast to the demands of LLM serving, where requests vary widely in size, latency sensitivity, and tenant requirements. Consequently, much of the burden of designing finer-grained scheduling policies is shifted to system developers. These observations underscore the need for research on more adaptive and workload-sensitive scheduling approaches that extend beyond FIFO and basic fairness.

Table 3. Scheduling and Execution Capabilities of Leading Open-Source LLM Inference Frameworks

Capability	vLLM	Aibrix	TGI (HF)	DeepSpeed-Inference	Llumnix
Token-level Scheduling	Yes	Yes	No	No	Yes
Dynamic Batching	Yes	Yes	Yes	Partial	Yes
Prioritization	Partial	Yes	Partial	No	Yes
Fair Queuing	No	Yes	No	No	Partial
Preemption	No	Partial	No	No	Partial
KV Cache-aware	Yes	Yes	Partial	Partial	Yes
Multi-tenancy	Partial	Yes	Partial	No	Partial
Auto-scaling	No	Partial	Yes	No	No
Concurrency Control	Yes	Yes	Yes	Partial	Partial
Scheduling Policy	FIFO (default)	Fair, FIFO	FIFO	Backend-defined	Fair (custom)

7.2 Industry Practices in Scheduling

Scheduling LLM inference efficiently remains a significant challenge due to the variability in request sizes, dynamic resource demands, and latency-sensitive applications. High-throughput systems must balance computational efficiency with response times while handling fluctuating user traffic. Companies deploying large-scale LLM services invest heavily in dynamic batching, priority scheduling, and load balancing techniques to ensure optimal performance [71].

OpenAI's deployment of GPT-based models [107] highlights the importance of dynamic request batching and adaptive scheduling[74]. To serve millions of requests daily, OpenAI employs techniques such as speculative decoding and response streaming, which enhance user experience by reducing perceived latency. Additionally, the use of fine-tuned routing algorithms [15] ensures that different tiers of customers receive appropriate response times, balancing cost and service quality[21].

Meta's LLaMA [139, 140] deployment provides insights into optimizing inference at scale. Unlike OpenAI's cloud-based approach, Meta emphasizes on-premises LLM serving, leveraging custom accelerators and optimized scheduling. Their resource-aware inference pipeline [102] enables fine-grained control over hardware utilization, allowing models to scale efficiently across diverse workloads [50]. Meta also explores multi-tenancy inference, which reduces overhead by sharing computational resources across different applications[110].

Google's LLM scheduling strategy incorporates TPU-based optimizations and auto-scaling inference clusters[1, 72]. The Gemini model [133] is deployed using a combination of serverless inference layers and dedicated model shards, enabling fine-tuned scaling depending on demand. Google's extensive experience in search and conversational AI [2, 138] underscores the necessity of asynchronous processing pipelines and real-time load balancing for sustaining high-availability services[16, 29].

Huawei has made significant strides in AI model serving, particularly in edge-cloud hybrid inference. The company's MindSpore AI framework [58, 155] enables efficient deployment of LLMs across cloud and edge devices, reducing latency for real-time applications. Huawei leverages heterogeneous computing with its Ascend AI chips [31, 90], optimizing scheduling to balance compute-intensive tasks between cloud servers and local devices[85]. This approach enhances responsiveness and reduces cloud dependency, making it a viable strategy for regions with limited internet infrastructure.

Table 4. Summary of Open-Source LLM Serving Frameworks and Industry Scheduling Practices

Name / Company	Key Features	Scheduling / Optimization Strategies
Open-Source Frameworks		
vLLM	Paged attention, high throughput, flexible batching	Token-aware scheduling, memory-optimized context handling, latency reduction
TensorRT-LLM (NVIDIA)	Tensor parallelism, GPU-level quantization, CUDA integration	Kernel fusion, graph-level optimizations, throughput-first design
DeepSpeed-Inference	Multi-GPU inference, Hugging Face/PyTorch integration	Model parallelism, tensor slicing, cost-efficient scaling
Llumnix	Adaptive caching, low-latency focus, dynamic batching	Load-aware scheduling, real-time prioritization for interactive use
TGI, FasterTransformer, vLLM	Quantization, CUDA kernels, transformer-specialized code	Dynamic batching, stream scheduling, inference acceleration
Industry Practices		
OpenAI	GPT deployment at scale	Dynamic batching, speculative decoding, response streaming, tier-aware routing
Meta (LLaMA)	On-premise focus, custom accelerators	Resource-aware pipelines, multi-tenancy, hardware-adaptive load distribution
Google (Gemini)	TPU deployment, serverless scaling, sharded inference	Auto-scaling clusters, async pipelines, real-time load balancing
Huawei (MindSpore)	Edge-cloud hybrid inference, Ascend chip integration	Heterogeneous scheduling, latency-aware offloading, regional fault tolerance

7.3 Scheduling Techniques per LLM Business Model

The diversity of commercial LLM offerings, ranging from token-based APIs to dedicated server rentals, introduces fundamentally different constraints on inference scheduling. Each business model shapes expectations around latency, throughput, fairness, and cost, which in turn drive the design of control and data plane mechanisms. In this section, we

examine how scheduling strategies adapt to the operational requirements and SLOs of common LLM delivery models, highlighting specialized optimizations for each.

7.3.1 Pay-Per-Token or Pay-Per-Request (API Usage). In the pay-per-token or pay-per-request business model, users are billed based on the number of tokens processed or the total number of API calls. This setup requires infrastructure capable of delivering low-latency, reliable responses while maintaining efficient resource utilization.

Request scheduling in this context must prioritize near real-time handling to meet user expectations. Stateless, event-driven architectures are typically employed [82], where incoming requests are immediately queued and dispatched to available model workers. To handle fluctuations in demand—such as diurnal patterns or unexpected traffic spikes—auto-scaling mechanisms dynamically adjust the number of active model-serving instances based on metrics like queue length, GPU utilization, or token throughput. In practice, production Machine Learning as a Service (MLaaS) platforms observe highly variable, latency-sensitive workloads, requiring fine-grained scheduling strategies to maintain efficiency and meet user SLOs [147]. Recent work has proposed cost-optimization strategies for token-based LLM usage, including dynamic early stopping and response truncation, to reduce compute overhead while preserving response quality [120].

Sections 7.3.6 and 7.3.7 (DaaS and Chatbots, respectively) can be viewed as special cases of this API-based model when their services are monetized on a per-request basis. While they exhibit distinct usage patterns and SLO requirements, in practice they may be co-located and routed through the same scheduling infrastructure described here.

Load balancing [156] is critical for distributing incoming requests evenly across model instances to prevent bottlenecks and optimize resource use. Effective designs often combine transport-layer and application-layer load balancing to maximize performance and observability. In parallel, rate limiting [119] enforces fair usage policies and protects the system from abuse. These limits are typically integrated with user authentication, applying quotas that cap the request rate or token consumption per user or API key, often tiered according to subscription plans.

7.3.2 Subscription-Based Access. In a subscription-based access model, users are granted varying levels of service depending on their subscription tier, which introduces the need for differentiated resource allocation and scheduling policies. Access control mechanisms ensure that each user’s requests are routed according to their entitlement, with premium tiers often receiving prioritized access to high-performance models or reserved capacity to guarantee low-latency responses even during peak periods. Surveyed MaaS platforms often adopt tiered subscriptions with guarantees around latency, capacity, or model quality [41].

To maintain fairness and manage load, time-based scheduling can also be implemented. For example, lower-tier subscribers may experience limited access or reduced service quality during periods of high demand, while higher-tier users retain full privileges. This necessitates the use of priority queues, where requests are scheduled based on user tier and SLOs. In some cases, deferred processing or scheduled access windows may be employed to optimize resource utilization while maintaining contractual obligations.

7.3.3 Server Rent-Based Access. In the server rent-based access model, users pay for dedicated server or GPU/TPU time, often in predefined blocks or on an on-demand basis. This model requires robust mechanisms for both fixed and dynamic resource allocation. For fixed resource allocation, users reserve compute resources in advance for specified durations. The system implements job queuing to manage overlapping requests, ensuring fair access and avoiding resource contention. Resource isolation (e.g., via containerization or VM partitioning) is also critical to guarantee performance and security for each tenant. Dynamic allocation introduces a self-service platform where users can schedule and manage their own resource bookings based on current availability. This requires real-time tracking

of resource usage and predictive scheduling to maximize utilization. Integration with cloud APIs or orchestration platforms, such as Kubernetes or Slurm, is typically used to automate provisioning and de-provisioning of hardware resources as user sessions begin and end.

7.3.4 On-Demand Model Hosting with Customization. In the on-demand hosting model, users deploy their own models, often with requirements for fine-tuning or other customizations. This introduces scheduling challenges around model loading, initialization, and ongoing resource allocation.

Model deployment scheduling must account for the time and resources needed to load large models into memory and apply custom configurations. Scheduling policies may prioritize certain users or models based on factors such as SLO agreements, resource availability, or client tier. Efficient orchestration is essential to minimize downtime between user requests and ensure smooth model rollouts, particularly when fine-tuning steps are involved.

To optimize cost and flexibility, the system can offer a mix of preemptible and reserved compute options. Preemptible instances provide lower-cost, short-term compute that can be reclaimed when demand spikes, making them suitable for experimentation or transient tasks. Reserved instances, by contrast, ensure guaranteed availability for long-term hosting of production models, offering users stability at a premium price point.

7.3.5 Enterprise SaaS Offering. In an enterprise Software as a Service (SaaS) model, the system must support multiple customers concurrently, often across diverse organizational needs and usage patterns. This requires robust multi-user and multi-tenant scheduling mechanisms to ensure fairness, isolation, and efficiency.

Multi-user scheduling focuses on allocating compute and memory resources to handle requests from different users within and across customer organizations. User isolation (achieved through techniques like containerization or namespace segmentation) is essential to prevent data leakage and performance interference between users. Resource partitioning ensures that no single user or team monopolizes shared infrastructure.

Multi-tenant scheduling extends this by managing LLM service provisioning at the tenant (organization) level. The system allocates resources according to tenant-specific quotas, subscription tiers, or negotiated SLOs. This often involves dynamic scaling and prioritization strategies to balance workloads between high- and low-priority tenants, such as selectively prioritizing response length or routing to lower-cost model variants, while enforcing strict usage limits and monitoring to maintain service integrity across all clients [120, 147].

7.3.6 Data-Processing as a Service (DaaS). In the Data-Processing as a Service (DaaS) model [136], users submit large datasets for processing, typically involving offline or delayed computation. This setup necessitates robust scheduling mechanisms to manage high data throughput and ensure reliable processing.

Batch processing scheduling is central to this model. Incoming data jobs are queued and processed in batches[10], with scheduling strategies that can be either time-based (e.g., nightly runs) or data-volume-based[6] (e.g., triggering a job once a threshold size is reached). Effective batch scheduling optimizes resource utilization and helps manage compute-intensive workloads without impacting real-time services.

Asynchronous processing is also a key component. Queue-based systems, such as Celery[137] or similar distributed task queues, enable the decoupling of data ingestion from processing, allowing the system to handle large volumes of data flexibly and resiliently. This architecture supports retries, failure handling, and load distribution, ensuring that data is processed reliably even in the face of transient errors or fluctuating demand.

7.3.7 LLM-Powered Chatbots as a Service. In the LLM-powered chatbots-as-a-service model, the system must handle a high volume of interactive, often unpredictable requests, with varying levels of complexity, latency sensitivity, and personalization. This creates distinct challenges for scheduling, load balancing, and memory management.

Request scheduling must efficiently manage bursty traffic patterns, particularly during peak usage windows or viral campaigns. This is done by dynamically prioritizing requests based on user criticality, query complexity, or SLOs. For instance, low-cost FAQ-style questions may require only shallow model computation, whereas multi-turn dialogues involving long-term context and reasoning (as seen in models like Meena [2] and LLaMA 2-Chat [140]) require more intensive resources and memory persistence across turns.

Load balancing plays a central role in sustaining responsiveness and availability under load. Traditional round-robin or static placement strategies often fall short when request heterogeneity is high. Advanced cluster-level routers and instance-level schedulers dynamically dispatch chatbot queries to specialized model instances based on current load, thermal constraints, and task affinity. Systems like Mooncake [114] propose KV cache-centric serving architectures that offload context storage from compute pipelines, enabling faster prefill processing and lowering decoding latency—particularly beneficial for long-context chat sessions.

These architectures must also optimize for cost efficiency, which often requires co-locating inference for multiple tenants or adapting scheduling windows in real time. Multi-tenant-aware batching, routing, and speculative decoding mechanisms are increasingly being deployed to meet both latency and utilization goals.

8 Comparison of Scheduling Approaches for LLM Inference

Scheduling strategies for LLM inference must navigate a complex trade-off space shaped by latency, resource efficiency, scalability, and cost. As workloads grow in size and variability, systems must adopt increasingly sophisticated mechanisms to meet performance targets and SLOs. In this section, we compare core scheduling approaches along three dimensions: latency reduction (e.g., cold start mitigation, HOL blocking), GPU utilization and cost-efficiency, and scalability to large or dynamic workloads.

8.1 Latency Reduction: Techniques for Minimizing Cold Start and HOL Blocking

Reducing inference latency in LLM systems requires addressing two primary bottlenecks: cold start delays and HOL blocking. Cold start delays arise when a model or service instance must be initialized before serving can begin, introducing substantial overhead due to container startup, model loading, and GPU memory allocation. HOL blocking occurs when earlier, high-cost inference requests stall the processing of shorter ones queued behind them.

A recent systematic review of cold start latency in serverless environments categorizes mitigation strategies into warm pooling, predictive prewarming, lazy loading, and function reuse [46]. These strategies are directly applicable to LLM-serving systems, where just-in-time resource provisioning can degrade user-perceived latency [39, 53]. Warm pools or idle model replicas can minimize cold starts at the expense of resource over-provisioning, while predictive prewarming aims to selectively activate model instances based on traffic forecasts. Recent scaling acceleration techniques including live migration and checkpoint warm-up significantly reduce cold start latency [128, 165, 169]. Lazy loading defers certain operations (e.g., tokenizer init or model weight transfer) until absolutely necessary, reducing initialization overhead for lighter requests.

In parallel, dynamic and adaptive batching techniques help mitigate HOL blocking by aggregating compatible inference requests based on arrival time and model parameters, improving GPU utilization while avoiding long stalls [7, 34]. Deep learning inference frameworks such as vLLM and Triton Inference Server implement these batching

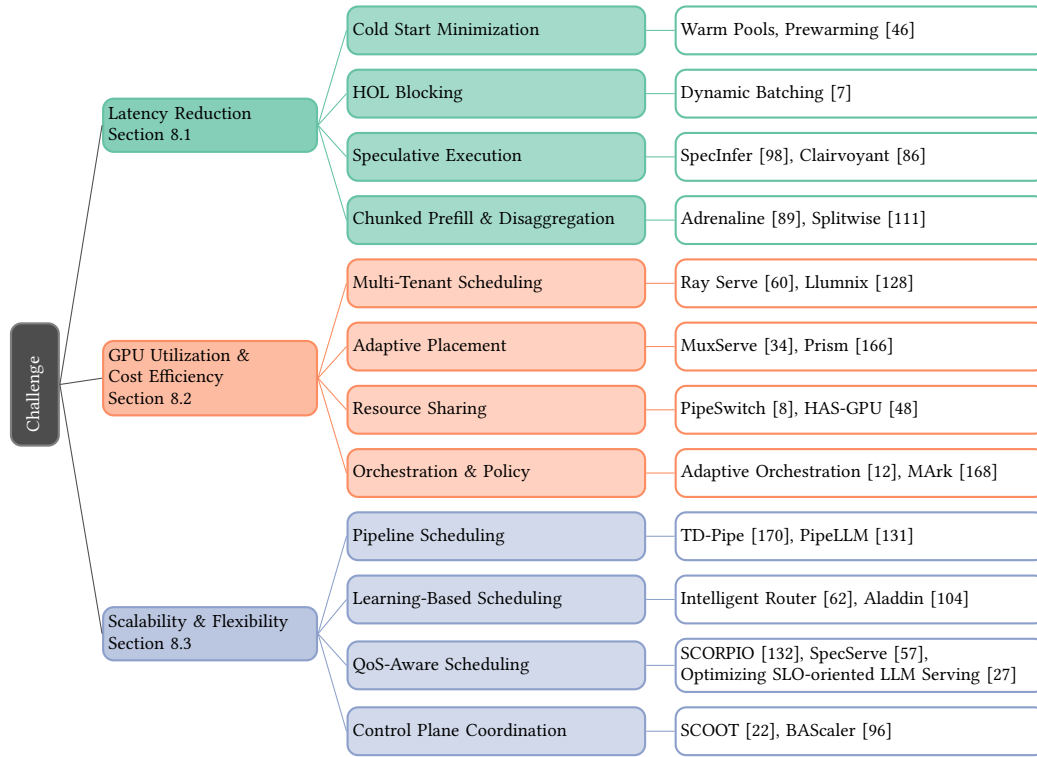


Fig. 6. Summary of Challenges and Key References

mechanisms to optimize throughput under variable traffic patterns [23, 93]. Continuous batching extends this by keeping an active queue of ready-to-process requests, reducing startup delays but requiring tighter scheduling control [164].

Speculative execution and prefetching have also been explored as latency reduction strategies, where potential future requests are preloaded or precomputed in parallel to mask wait times. Recent systems such as Semi-Clairvoyant Scheduling [86] and SpecInfer [98] refine this approach using partial request prediction and verification-driven speculative decoding, achieving both latency reduction and compute efficiency. While effective, these methods introduce extra computational overhead and depend on accurate workload prediction. More recently, chunked-prefill scheduling has emerged as a promising technique for reducing latency in autoregressive generation, where long input sequences are divided into smaller chunks that can be processed in parallel, significantly reducing time-to-first-token. Disaggregated serving designs such as Adrenaline [89], Splitwise [111], and DistServe [179] further enhance latency performance by executing the prefill and decode stages on decoupled backends, enabling memory reuse and pipelined inference.

8.2 GPU Utilization & Cost Trade-offs: Balancing Performance vs. Efficiency

Optimizing GPU utilization while maintaining cost efficiency requires a delicate balance between scheduling complexity, computational overhead, and model performance. Static batching offers predictable execution patterns and is straightforward to implement, but it struggles to adapt to fluctuating workloads, leading to inefficient resource usage.

More advanced strategies, such as model parallelism-aware scheduling, distribute inference tasks across multiple GPUs, allowing for efficient execution of large-scale LLMs. However, the increased inter-GPU communication overhead can lead to performance bottlenecks.

Multi-tenant scheduling and load balancing techniques improve overall hardware utilization by dynamically allocating resources based on demand. These approaches are particularly useful in cloud-based LLM APIs, where multiple users compete for computational resources. Frameworks like Ray Serve [60] and Llumnix [128] implement sophisticated load balancing mechanisms to prevent resource starvation while maintaining fairness across multiple tenants. Recent systems such as MuxServe [34] and Prism [166] further enhance utilization through adaptive placement, batching, and memory-aware request scheduling.

Resource-sharing policies such as those used in PipeSwitch [8] and HAS-GPU [48] enable fine-grained GPU partitioning, allowing multiple inference tasks to safely share GPU cores or streaming multiprocessors. These techniques reduce idle cycles and allow the scheduler to mix latency-sensitive and throughput-optimized workloads. Meanwhile, adaptive orchestration frameworks like Adaptive Orchestration [12] and MArk [168] explicitly optimize the trade-off between cost and performance by selecting execution strategies based on current load, SLO targets, and infrastructure pricing.

8.3 Scalability & Flexibility: Adapting to Increasing LLM Sizes

The scalability of scheduling strategies is crucial as LLMs continue to grow in complexity and computational requirements. Pipeline scheduling enhances parallelism by dividing inference tasks into sequential stages, thereby reducing processing bottlenecks and improving overall throughput. Systems like TD-Pipe [170] and PipeLLM [131] adopt temporally disaggregated and speculative pipelining strategies to enable concurrent execution across multiple model stages or servers, significantly improving goodput and latency for large models.

Learning-based scheduling and learning-to-rank approaches have emerged as adaptive techniques capable of dynamically adjusting scheduling decisions based on real-time workload patterns. Intelligent Router [62] and Aladdin [104] leverage reinforcement learning to optimize query routing and resource allocation, while AdaServe [88] applies multi-SLO-aware placement at cluster scale. These methods improve responsiveness in heterogeneous environments, but their performance depends on training quality and generalization to diverse workload distributions. Control-plane frameworks like SCOOT [22] and BAScaler [96] complement these approaches by forecasting workload trends and pruning suboptimal scaling plans, offering greater adaptability in bursty or evolving environments.

QoS-aware scheduling and preemption mechanisms are essential for ensuring compliance with SLOs in enterprise AI applications. Systems like SCORPIO [132], SpecServe [57], and Optimizing SLO-oriented LLM Serving [27] implement SLO-aware batching, speculative execution control, and deadline-based prioritization to guarantee service quality under load. Similarly, WindServe [37] and Arrow [151] dynamically adjust scheduling strategies based on latency-throughput trade-offs. Fluid Scheduling [7], a control-plane technique based on queueing fluid models, adapts wait windows and concurrency levels to meet real-time performance targets.

Scalability also requires infrastructure-level flexibility to adapt to shifting workloads and resource availability. Adaptive Orchestration [12] coordinates inference across heterogeneous accelerators, balancing cost and performance. Prism [166] introduces a two-level scheduler for efficient resource sharing across multiple LLMs, and DeepFlow [54] manages multi-stage pipeline alignment during autoscaling. HAS-GPU [48] supports vertical elasticity by reallocating GPU streaming multiprocessors (SMs), enabling fine-grained scale-up under tight hardware budgets.

The choice of scheduling strategy for LLM inference depends on multiple factors, including latency requirements, GPU availability, and workload variability. For applications requiring low-latency responses, adaptive batching, preemption,

and chunked-prefill scheduling provide effective solutions. Cost-efficient inference can be achieved through multi-tenant scheduling, load balancing, and resource-sharing policies. Large-scale LLM deployments benefit from pipeline scheduling and model-parallelism-aware techniques, which enable efficient execution across multiple GPUs. Finally, learning-based and control-plane-guided scheduling methods offer promising adaptability for unpredictable workloads but require robust monitoring and forecasting capabilities to avoid inefficiencies.

9 Open Challenges and Future Directions

As LLM inference systems mature, new capabilities in scheduling, autoscaling, and resource management have unlocked substantial improvements in responsiveness, efficiency, and scalability. Yet, several open challenges remain that span the architecture stack. We identify four key themes that shape the future research agenda in this space.

9.1 Toward Predictive and Fine-Grained Autoscaling

Despite progress in horizontal and vertical autoscaling, today’s systems still suffer from provisioning delays, cold-start overheads, and coarse-grained scaling units—especially in tensor-parallel or disaggregated settings where coordination across multi-GPU groups is required. Reactive scaling decisions often rely on conservative heuristics and cooldown policies that hinder responsiveness during bursty demand. Future autoscaling approaches must evolve into closed-loop control systems that integrate workload forecasting, dispatch logic, and model residency constraints. Promising directions include reinforcement learning-based policies, adaptive cooldown mechanisms, and finer-grained scaling primitives such as expert or adapter-level activation. Effective autoscaling will require tight coordination with the scheduler and model manager to optimize for cost, responsiveness, and stability.

9.2 Adaptive and Multi-Objective Model Management

Model management is increasingly critical in multi-model and multi-tenant serving platforms. While demand-driven loading, cache-aware eviction, and memory pinning have mitigated cold-start latency, current policies remain largely reactive and memory-centric. Future systems must expand beyond simple residency decisions to account for compute contention, I/O bottlenecks, and workload-specific QoS needs. This calls for holistic, multi-resource scheduling frameworks that reason across latency, throughput, and SLO satisfaction. Learning-based techniques such as bandits and reinforcement learning may help drive adaptive model retention strategies that evolve with traffic patterns. Ultimately, the goal is to develop model management that is not just resource-efficient but workload-adaptive, QoS-aware, and system-integrated.

9.3 Cross-Layer Co-Design for Hardware and Runtime Efficiency

Emerging runtime systems are shifting from static execution pipelines toward adaptive, modular runtimes that dynamically adjust to memory pressure, token distribution, and hardware heterogeneity. Instance-level schedulers are optimizing token batching, KV cache reuse, and execution knobs to increase GPU utilization and reduce latency. Looking forward, future runtimes should integrate more deeply with autoscalers, KV managers, and cluster schedulers to enable full-stack optimization. Hardware heterogeneity further motivates co-design across memory allocation, scheduling granularity, and accelerator abstraction. Sub-device-level management (e.g., attention layer offloading, KV compression) may offer new levers for improving performance and efficiency under real-world constraints.

9.4 Unified Orchestration for Robust, SLO-Compliant Serving

Workload orchestration is evolving from static routing to adaptive, latency-sensitive coordination across distributed infrastructure. However, orchestration frameworks must now contend with fluctuating demand, heterogeneous deployments, and SLO diversity. The next generation of orchestrators will need tighter integration with autoscaling, failure remediation, request forecasting, and model lifecycle management. Hybrid schedulers that blend centralized optimization with decentralized responsiveness may offer a path forward. Additionally, future orchestration should incorporate cost-awareness, fairness, and energy efficiency as first-class objectives. Rich telemetry and workload modeling will be essential to anticipate demand and make proactive decisions across the control plane.

10 Conclusion

The rise of LLMs has intensified demand for efficient and adaptable inference scheduling. This survey organized the landscape across control and data planes, highlighting emerging patterns and trade-offs in modern LLM serving systems across a corpus of 180 references. Control-plane innovations emphasize SLO-aware orchestration, predictive autoscaling, and fine-grained GPU resource management. On the data plane, schedulers are being redesigned for multi-LLM colocation, speculative execution, and KV cache reuse to handle bursty, heterogeneous workloads.

Looking ahead, inference scheduling will move toward unified, cross-layer frameworks that integrate scheduling, scaling, and caching into learned, end-to-end optimization loops. As deployment environments diversify, adaptive, hardware-agnostic, and fair scheduling will be essential for cost-effective, scalable AI. Ultimately, LLM inference scheduling is a core systems challenge and a key enabler of accessible and sustainable AI at scale.

References

- [1] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: a system for large-scale machine learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation* (Savannah, GA, USA) (OSDI'16). USENIX Association, USA, 265–283.
- [2] Daniel Adiwardana, Minh-Thang Luong, David R So, Jamie Hall, Noah Fiedel, Romal Thoppilan, Zi Yang, Apoorv Kulshreshtha, Gaurav Nemade, Yifeng Lu, et al. 2020. Towards a human-like open-domain chatbot. *arXiv preprint arXiv:2001.09977* (2020).
- [3] Saurabh Agarwal, Anyong Mao, Aditya Akella, and Shivaram Venkataraman. 2024. SYMPHONY: Improving Memory Management for LLM Inference Workloads. *CoRR* abs/2412.16434 (2024). <https://doi.org/10.48550/arXiv.2412.16434>
- [4] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. VIDUR: A LARGE-SCALE SIMULATION FRAMEWORK FOR LLM INFERENCE. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6. 351–366. https://proceedings.mlsys.org/paper_files/paper/2024/file/b74a8de47d2b3c928360e0a011f48351-Paper-Conference.pdf
- [5] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 117–134. <https://www.usenix.org/conference/osdi24/presentation/agrawal>
- [6] Syed Muhammad Abrar Akber, Imran Ali Khan, Syed Shah Muhammad, Syed Muhammad Mohsin, Iftikhar Ahmed Khan, Shahaboddin Shamshirband, and Anthony T. Chronopoulos. 2018. Data Volume Based Data Gathering in WSNs using Mobile Data Collector. In *Proceedings of the 22nd International Database Engineering & Applications Symposium* (Villa San Giovanni, Italy) (IDEAS '18). Association for Computing Machinery, New York, NY, USA, 199–207. doi:10.1145/3216122.3216166
- [7] Ruicheng Ao, Gan Luo, David Simchi-Levi, and Xinshang Wang. 2025. Optimizing LLM Inference: Fluid-Guided Online Scheduling with Memory Constraints. *arXiv:2504.11320 [cs.LG]* <https://arxiv.org/abs/2504.11320>
- [8] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. 2020. PipeSwitch: fast pipelined context switching for deep learning applications. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation (OSDI'20)*. USENIX Association, USA, Article 28, 16 pages.
- [9] Agrim Bari, Parikshit Hegde, and Gustavo de Veciana. 2025. Optimal Scheduling Algorithms for LLM Inference: Theory and Practice. *arXiv:2508.01002 [cs.LG]* <https://arxiv.org/abs/2508.01002>
- [10] Sarah Benjelloun, Mohamed El Mehdi El Aissi, Yassine Loukili, Younes Lakhri, Safae Elhaj Ben Ali, Hiba Chougrad, and Abdessamad El Boushaki. 2020. Big Data Processing: Batch-based processing and stream-based processing. In *2020 Fourth International Conference On Intelligent Computing*

- in *Data Sciences (ICDS)*. 1–6. doi:10.1109/ICDS50568.2020.9268684
- [11] Kyungmin Bin, Seungbeom Choi, Jimyoung Son, Jieun Choi, Daseul Bae, Daehyeon Baek, Kihyo Moon, Minsung Jang, and Hyojung Lee. 2025. FineServe: Precision-Aware KV Slab and Two-Level Scheduling for Heterogeneous Precision LLM Serving. arXiv:2509.06261 [cs.DC] <https://arxiv.org/abs/2509.06261>
 - [12] Yahav Biran and Imry Kissos. 2025. Adaptive Orchestration for Large-Scale Inference on Heterogeneous Accelerator Systems Balancing Cost, Performance, and Resilience. arXiv:2503.20074 [cs.PF] <https://arxiv.org/abs/2503.20074>
 - [13] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. 2021. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* (2021).
 - [14] Wan Borui, Zhao Juntao, Jiang Chenyu, Guo Chuanxiong, and Wu Chuan. 2025. Efficient LLM Serving on Hybrid Real-time and Best-effort Requests. arXiv:2504.09590 [cs.AI] <https://arxiv.org/abs/2504.09590>
 - [15] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
 - [16] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. 2016. Borg, omega, and kubernetes. *Commun. ACM* 59, 5 (2016), 50–57.
 - [17] Shi Chang, Boyuan Chen, Kishanthan Thangarajah, Hanan Lutfiyya, and Ahmed E. Hassan. 2025. Adaptive Request Scheduling for CodeLLM Serving with SLA Guarantees. arXiv:2506.19677 [cs.SE] <https://arxiv.org/abs/2506.19677>
 - [18] Himani Chaudhary, Geetanjali Sharma, Dinesh Kumar Nishad, and Saifullah Khalid. 2025. Advanced queueing and scheduling techniques in cloud computing using AI-based model order reduction. *Discover Computing* 28, 1 (2025), 1–40.
 - [19] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318* (2023).
 - [20] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. Frugalpvt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176* (2023).
 - [21] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
 - [22] Ke Cheng, Zhi Wang, Wen Hu, Tiannuo Yang, Jianguo Li, and Sheng Zhang. 2025. SCOOT: SLO-Oriented Performance Tuning for LLM Inference Engines. arXiv:2408.04323 [cs.DC] <https://arxiv.org/abs/2408.04323>
 - [23] Krishna Teja Chitty-Venkata, Siddhisanket Raskar, Bharat Kale, Farah Ferdaus, Aditya Tanikanti, Ken Raffanetti, Valerie Taylor, Murali Emani, and Venkatram Vishwanath. 2025. LLM-Inference-Bench: Inference Benchmarking of Large Language Models on AI Accelerators. In *Proceedings of the SC '24 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis (Atlanta, GA, USA) (SC-W '24)*. IEEE Press, 1362–1379. doi:10.1109/SCW63240.2024.00178
 - [24] Peizhuang Cong, Qizhi Chen, Haochen Zhao, and Tong Yang. 2025. BATON: Enhancing Batch-wise Inference Efficiency for Large Language Models via Dynamic Re-batching. In *Proceedings of the ACM on Web Conference 2025 (Sydney NSW, Australia) (WWW '25)*. Association for Computing Machinery, New York, NY, USA, 2309–2318. doi:10.1145/3696410.3714950
 - [25] Patrick H. Coppock, Brian Zhang, Eliot H. Solomon, Vasilis Kypriotis, Leon Yang, Bikash Sharma, Dan Schatzberg, Todd C. Mowry, and Dimitrios Skarlatos. 2025. LithOS: An Operating System for Efficient Machine Learning on GPUs. arXiv:2504.15465 [cs.OS] <https://arxiv.org/abs/2504.15465>
 - [26] Daniel Crankshaw, Gur-Eyal Sela, Xiangxi Mo, Corey Zumar, Ion Stoica, Joseph Gonzalez, and Alexey Tumanov. 2020. InferLine: latency-aware provisioning and scaling for prediction serving pipelines. In *Proceedings of the 11th ACM Symposium on Cloud Computing (Virtual Event, USA) (SoCC '20)*. Association for Computing Machinery, New York, NY, USA, 477–491. doi:10.1145/3419111.3421285
 - [27] Weihao Cui, Yukang Chen, Han Zhao, Ziyi Xu, Quan Chen, Xusheng Chen, Yangjie Zhou, Shixuan Sun, and Minyi Guo. 2025. Optimizing SLO-oriented LLM Serving with PD-Multiplexing. arXiv:2504.14489 [cs.OS] <https://arxiv.org/abs/2504.14489>
 - [28] Yinwei Dai, Rui Pan, Anand Iyer, Kai Li, and Ravi Netravali. 2024. Apparate: Rethinking Early Exits to Tame Latency-Throughput Tensions in ML Serving. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP '24)*. ACM, 607–623. doi:10.1145/3694715.3695963
 - [29] Jeffrey Dean and Luiz André Barroso. 2013. The tail at scale. *Commun. ACM* 56, 2 (2013), 74–80.
 - [30] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. doi:10.18653/v1/N19-1423
 - [31] Aritra Dhar, Clément Thorens, Lara Magdalena Lazier, and Lukas Cavigelli. 2024. Ascend-CC: Confidential Computing on Heterogeneous NPU for Emerging Generative AI Workloads. *arXiv preprint arXiv:2407.11888* (2024).
 - [32] Xiangjue Dong, Maria Teleki, and James Caverlee. 2024. A Survey on LLM Inference-Time Self-Improvement. arXiv:2412.14352 [cs.CL] <https://arxiv.org/abs/2412.14352>

- [33] Jiangsu Du, Hongbin Zhang, Taosheng Wei, Zhenyi Zheng, Kaiyi Wu, Zhiguang Chen, and Yutong Lu. 2025. EcoServe: Enabling Cost-effective LLM Serving with Proactive Intra- and Inter-Instance Orchestration. arXiv:2504.18154 [cs.DC] <https://arxiv.org/abs/2504.18154>
- [34] Jiangfei Duan, Runyu Lu, Haojie Duanmu, Xiuhong Li, Xingcheng Zhang, Dahua Lin, Ion Stoica, and Hao Zhang. 2024. MuxServe: flexible spatial-temporal multiplexing for multiple LLM serving. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML'24)*. JMLR.org, Article 473, 13 pages.
- [35] Hugging Face. 2023. Hugging Face Text Generation Inference. <https://github.com/huggingface/text-generation-inference>. Commit: 3c02262, Accessed on: 2025-05-09.
- [36] Ruibo Fan, Xiangrui Yu, Peijie Dong, Zeyu Li, Gu Gong, Qiang Wang, Wei Wang, and Xiaowen Chu. 2025. SpInfer: Leveraging Low-Level Sparsity for Efficient Large Language Model Inference on GPUs. In *Proceedings of the Twentieth European Conference on Computer Systems (Rotterdam, Netherlands) (EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 243–260. doi:10.1145/3689031.3717481
- [37] Jingqi Feng, Yukai Huang, Rui Zhang, Sicheng Liang, Ming Yan, and Jie Wu. 2025. WindServe: Efficient Phase-Disaggregated LLM Serving with Stream-based Dynamic Scheduling. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 1283–1295. doi:10.1145/3695053.3730999
- [38] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: low-latency serverless inference for large language models. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation (Santa Clara, CA, USA) (OSDI'24)*. USENIX Association, USA, Article 8, 19 pages.
- [39] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. {ServerlessLLM}:{Low-Latency} serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI'24)*. 135–153.
- [40] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. *arXiv preprint arXiv:2408.15792* (2024).
- [41] Wensheng Gan, Shicheng Wan, and Philip S. Yu. 2023. Model-as-a-Service (MaaS): A Survey. In *2023 IEEE International Conference on Big Data (BigData)*. 4636–4645. doi:10.1109/BigData59044.2023.10386351
- [42] Shihong Gao, Xin Zhang, Yanyan Shen, and Lei Chen. 2025. Apt-Serve: Adaptive Request Scheduling on Hybrid Cache for Scalable LLM Inference Serving. *Proceedings of the ACM on Management of Data* 3, 3 (June 2025), 1–28. doi:10.1145/3725394
- [43] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *CoRR* abs/2312.10997 (2023). <https://doi.org/10.48550/arXiv.2312.10997>
- [44] Talia Gershon, Seetharami Seelam, Brian Belgodere, Milton Bonilla, Lan Hoang, Danny Barnett, I Chung, Apoorve Mohan, Ming-Hung Chen, Lixiang Luo, et al. 2025. The infrastructure powering IBM's Gen AI model development. arXiv:2407.05467 [cs.DC] <https://arxiv.org/abs/2407.05467>
- [45] Kanishk Goel, Jayashree Mohan, Nipun Kwatra, Ravi Shreyas Anupindi, and Ramachandran Ramjee. 2025. Niyama : Breaking the Silos of LLM Inference Serving. arXiv:2503.22562 [cs.LG] <https://arxiv.org/abs/2503.22562>
- [46] Muhammed Golec, Gunet Kaur Walia, Mohit Kumar, Felix Cuadrado, Sukhpal Singh Gill, and Steve Uhlig. 2024. Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions. *ACM Comput. Surv.* 57, 3, Article 65 (Nov. 2024), 36 pages. doi:10.1145/3700875
- [47] Ruihao Gong, Shihao Bai, Siyu Wu, Yunqian Fan, Zaijun Wang, Xiuhong Li, Hailong Yang, and Xianglong Liu. 2025. Past-Future Scheduler for LLM Serving under SLA Guarantees. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (Rotterdam, Netherlands) (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 798–813. doi:10.1145/3676641.3716011
- [48] Jianfeng Gu, Puxuan Wang, Isaac Nunezand, Kai Huang, and Michael Gerndt. 2025. HAS-GPU: Efficient Hybrid Auto-scaling with Fine-grained GPU Allocation for SLO-aware Serverless Inferences. arXiv:2505.01968 [cs.DC] <https://arxiv.org/abs/2505.01968>
- [49] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large Language Model Based Multi-agents: A Survey of Progress and Challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, Kate Larson (Ed.). International Joint Conferences on Artificial Intelligence Organization, 8048–8057. doi:10.24963/ijcai.2024/890 Survey Track.
- [50] Kim Hazelwood, Sarah Bird, David Brooks, Soumith Chintala, Utku Diril, Dmytro Dzhulgakov, Mohamed Fawzy, Bill Jia, Yangqing Jia, Aditya Kalro, et al. 2018. Applied machine learning at facebook: A datacenter infrastructure perspective. In *2018 IEEE international symposium on high performance computer architecture (HPCA)*. IEEE, 620–629.
- [51] Ke Hong, Lufang Chen, Zhong Wang, Xiuhong Li, Qiuli Mao, Jianping Ma, Chao Xiong, Guanyu Wu, Buhe Han, Guohao Dai, Yun Liang, and Yu Wang. 2025. semi-PD: Towards Efficient LLM Serving via Phase-Wise Disaggregated Computation and Unified Storage. arXiv:2504.19867 [cs.CL] <https://arxiv.org/abs/2504.19867>
- [52] CunChen Hu, HeYang Huang, LiangLiang Xu, XuSheng Chen, Chenxi Wang, Jiang Xu, Shuang Chen, Hao Feng, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2025. ShuffleInfer: Disaggregate LLM Inference for Mixed Downstream Workloads. *ACM Trans. Archit. Code Optim.* (April 2025). doi:10.1145/3732941 Just Accepted.
- [53] Junhao Hu, Jiang Xu, Zhixia Liu, Yulong He, Yuetao Chen, Hao Xu, Jiang Liu, Jie Meng, Baoquan Zhang, Shining Wan, Gengyuan Dan, Zhiyu Dong, Zhihao Ren, Changhong Liu, Tao Xie, Dayun Lin, Qin Zhang, Yue Yu, Hao Feng, Xusheng Chen, and Yizhou Shan. 2025. DeepServe: Serverless Large Language Model Serving at Scale. arXiv:2501.14417 [cs.DC] <https://arxiv.org/abs/2501.14417>
- [54] Junhao Hu, Jiang Xu, Zhixia Liu, Yulong He, Yuetao Chen, Hao Xu, Jiang Liu, Baoquan Zhang, Shining Wan, Gengyuan Dan, et al. 2025. DeepFlow: Serverless Large Language Model Serving at Scale. *arXiv preprint arXiv:2501.14417* (2025).

- [55] Jianmin Hu, Minxian Xu, Kejiang Ye, and Chengzhong Xu. 2025. BrownoutServe: SLO-Aware Inference Serving under Bursty Workloads for MoE-based LLMs. *arXiv:2507.17133 [cs.DC]* <https://arxiv.org/abs/2507.17133>
- [56] Jinqi Huang, Yi Xiong, Xuebing Yu, Wenjie Huang, Entong Li, Li Zeng, and Xin Chen. 2025. SLO-Aware Scheduling for Large Language Model Inferences. *arXiv:2504.14966 [cs.DC]* <https://arxiv.org/abs/2504.14966>
- [57] Kaiyu Huang, Hao Wu, Zhubo Shi, Han Zou, Minchen Yu, and Qingjiang Shi. 2025. SpecServe: Efficient and SLO-Aware Large Language Model Serving with Adaptive Speculative Decoding. *arXiv:2503.05096 [cs.CL]* <https://arxiv.org/abs/2503.05096>
- [58] Ltd. Huawei Technologies Co. 2022. Huawei mindspore ai development framework. In *Artificial Intelligence Technology*. Springer, 137–162.
- [59] Nikoleta Iliakopoulou, Jovan Stojkovic, Chloe Alverti, Tianyin Xu, Hubertus Franke, and Josep Torrellas. 2024. Chameleon: Adaptive Caching and Scheduling for Many-Adapter LLM Inference Environments. *arXiv preprint arXiv:2411.17741* (2024).
- [60] Anyscale Inc. 2023. Ray Serve: Scalable Model Serving with Python. <https://docs.ray.io/en/latest/serve/>.
- [61] Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, Anoop Kulkarni, Steve Kofsky, and Saravan Rajmohan. 2025. Performance Aware LLM Load Balancer for Mixed Workloads. In *Proceedings of the 5th Workshop on Machine Learning and Systems* (World Trade Center, Rotterdam, Netherlands) (*EuroMLSys '25*). Association for Computing Machinery, New York, NY, USA, 19–30. doi:10.1145/3721146.3721947
- [62] Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, Anoop Kulkarni, Steve Kofsky, and Saravan Rajmohan. 2025. Intelligent Router for LLM Workloads: Improving Performance Through Workload-Aware Load Balancing. *arXiv:2408.13510 [cs.DC]* <https://arxiv.org/abs/2408.13510>
- [63] Rajendra K Jain, Dah-Ming W Chiu, William R Hawe, et al. 1984. A quantitative measure of fairness and discrimination. *Eastern Research Laboratory, Digital Equipment Corporation, Hudson, MA* 21, 1 (1984), 2022–2023.
- [64] Shashwat Jaiswal, Kunal Jain, Yogesh Simmhan, Anjaly Parayil, Ankur Mallick, Rujia Wang, Renee St. Amant, Chetan Bansal, Victor Rühle, Anoop Kulkarni, Steve Kofsky, and Saravan Rajmohan. 2025. Serving Models, Fast and Slow: Optimizing Heterogeneous LLM Inferencing Workloads at Scale. *arXiv:2502.14617 [cs.DC]* <https://arxiv.org/abs/2502.14617>
- [65] Suhas Jayaram Subramanya, Daiyaan Arfeen, Shouxu Lin, Aurick Qiao, Zhihao Jia, and Gregory R. Ganger. 2023. Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (*SOSP '23*). Association for Computing Machinery, New York, NY, USA, 642–657. doi:10.1145/3600006.3613175
- [66] Jinwoo Jeong, Seungsu Baek, and Jeongseob Ahn. 2023. Fast and Efficient Model Serving Using Multi-GPUs with Direct-Host-Access. In *Proceedings of the Eighteenth European Conference on Computer Systems* (Rome, Italy) (*EuroSys '23*). Association for Computing Machinery, New York, NY, USA, 249–265. doi:10.1145/3552326.3567508
- [67] Chaoyi Jiang, Lei Gao, Hossein Entezari Zarch, and Murali Annavaram. 2024. Efficient LLM Inference with I/O-Aware Partial KV Cache Recomputation. *CoRR* abs/2411.17089 (2024). <https://doi.org/10.48550/arXiv.2411.17089>
- [68] Yibo Jin, Tao Wang, Huimin Lin, Mingyang Song, Peiyang Li, Yipeng Ma, Yicheng Shan, Zhengfan Yuan, Cailong Li, Yajing Sun, et al. 2024. P/d-serve: Serving disaggregated large language model at scale. *arXiv preprint arXiv:2408.08147* (2024).
- [69] Yibo Jin, Tao Wang, Huimin Lin, Mingyang Song, Peiyang Li, Yipeng Ma, Yicheng Shan, Zhengfan Yuan, Cailong Li, Yajing Sun, Tiandeng Wu, Xing Chu, Ruizhi Huan, Li Ma, Xiao You, Wenting Zhou, Yunpeng Ye, Wen Liu, Xiangkun Xu, Yongsheng Zhang, Tiantian Dong, Jiawei Zhu, Zhe Wang, Xijian Ju, Jianxun Song, Haoliang Cheng, Xiaojing Li, Jiantong Ding, Hefei Guo, and Zhengyong Zhang. 2024. P/D-Serve: Serving Disaggregated Large Language Model at Scale. *arXiv:2408.08147 [cs.DC]* <https://arxiv.org/abs/2408.08147>
- [70] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S3: increasing GPU utilization during generative inference for higher throughput. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (*NIPS '23*). Curran Associates Inc., Red Hook, NY, USA, Article 791, 13 pages.
- [71] Yihong Jin and Ze Yang. 2025. Scalability Optimization in Cloud-Based AI Inference Services: Strategies for Real-Time Load Balancing and Automated Scaling. *arXiv preprint arXiv:2504.15296* (2025).
- [72] Norman P Jouppe, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [73] Aditya K. Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. POD-Attention: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) (*ASPLOS '25*). Association for Computing Machinery, New York, NY, USA, 897–912. doi:10.1145/3676641.3715996
- [74] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [75] Redwan Ibne Seraj Khan, Kunal Jain, Haiying Shen, Ankur Mallick, Anjaly Parayil, Anoop Kulkarni, Steve Kofsky, Pankhuri Choudhary, Renée St. Amant, Rujia Wang, Yue Cheng, Ali R. Butt, Victor Rühle, Chetan Bansal, and Saravan Rajmohan. 2024. Ensuring Fair LLM Serving Amid Diverse Applications. *arXiv:2411.15997 [cs.LG]* <https://arxiv.org/abs/2411.15997>
- [76] Hyungyo Kim, Nachuan Wang, Qirong Xia, Jinghan Huang, Amir Yazdanbakhsh, and Nam Sung Kim. 2025. LIA: A Single-GPU LLM Inference Acceleration with Cooperative AMX-Enabled CPU-GPU Computation and CXL Offloading. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture* (*ISCA '25*). Association for Computing Machinery, New York, NY, USA, 544–558. doi:10.1145/3695053.3731092

- [77] Kyoungmin Kim, Kijae Hong, Caglar Gulcehre, and Anastasia Ailamaki. 2025. Optimizing LLM Inference for Database Systems: Cost-Aware Scheduling for Concurrent Requests. *arXiv:2411.07447 [cs.PF]* <https://arxiv.org/abs/2411.07447>
- [78] Young Geun Kim and Carole-Jean Wu. 2020. Autoscale: Energy efficiency optimization for stochastic edge inference using reinforcement learning. In *2020 53rd Annual IEEE/ACM international symposium on microarchitecture (MICRO)*. IEEE, 1082–1096.
- [79] Ferdi Kossmann, Bruce Fontaine, Daya Khudia, Michael Cafarella, and Samuel Madden. 2025. Is the GPU Half-Empty or Half-Full? Practical Scheduling Techniques for LLMs. *arXiv:2410.17840 [cs.LG]* <https://arxiv.org/abs/2410.17840>
- [80] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. 611–626.
- [81] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv:2309.06180 [cs.LG]* <https://arxiv.org/abs/2309.06180>
- [82] Vincent Lannuri, Laurent D'Orazio, Olivier Barais, and Jalil Boukhobza. 2023. Serverless Cloud Computing: State of the Art and Challenges. In *Serverless Computing: Principles and Paradigms*, Rajalakshmi Krishnamurthi, Adarsh Kumar, Sukhpal Singh Gill, and Rajkumar Buyya (Eds.). Springer International Publishing, Cham, 275–316. doi:10.1007/978-3-031-26633-1_11
- [83] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. *arXiv:2406.19707 [cs.LG]* <https://arxiv.org/abs/2406.19707>
- [84] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. 2024. Llm inference serving: Survey of recent advances and opportunities. *arXiv preprint arXiv:2407.12391 (2024)*.
- [85] En Li, Liekang Zeng, Zhi Zhou, and Xu Chen. 2020. Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing. *IEEE Transactions on Wireless Communications* 19, 1 (2020), 447–457. doi:10.1109/TWC.2019.2946140
- [86] Ruixiao Li, Fahao Chen, and Peng Li. 2025. Semi-Clairvoyant Scheduling of Speculative Decoding Requests to Minimize LLM Inference Latency. *arXiv:2505.17074 [cs.CL]* <https://arxiv.org/abs/2505.17074>
- [87] Suyi Li, Hanfeng Lu, Tianyuan Wu, Minchen Yu, Qizhen Weng, Xusheng Chen, Yizhou Shan, Binhang Yuan, and Wei Wang. 2025. TOPPINGS: CPU-Assisted, Rank-Aware Adapter Serving for LLM Inference. In *Proc. USENIX ATC*.
- [88] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, Zeyu Wang, Qinghan Chen, Shuhuai Lin, April Yang, Zhihao Zhang, Zhuoming Chen, Sean Lai, Xinhao Cheng, Xupeng Miao, and Zhihao Jia. 2025. AdaServe: Accelerating Multi-SLO LLM Serving with SLO-Customized Speculative Decoding. *arXiv:2501.12162 [cs.CL]* <https://arxiv.org/abs/2501.12162>
- [89] Yunkai Liang, Zhangyu Chen, Pengfei Zuo, Zhi Zhou, Xu Chen, and Zhou Yu. 2025. Injecting Adrenaline into LLM Serving: Boosting Resource Utilization and Throughput via Attention Disaggregation. *arXiv:2503.20552 [cs.DC]* <https://arxiv.org/abs/2503.20552>
- [90] Heng Liao, Jiajin Tu, Jing Xia, Hu Liu, Xiping Zhou, Honghui Yuan, and Yuxing Hu. 2021. Ascend: a scalable and unified architecture for ubiquitous deep neural network computing: Industry track paper. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 789–801.
- [91] Qunyou Liu, Darong Huang, Marina Zapater, and David Atienza. 2025. GreenLLM: SLO-Aware Dynamic Frequency Scaling for Energy-Efficient LLM Serving. *arXiv:2508.16449 [cs.PF]* <https://arxiv.org/abs/2508.16449>
- [92] Yuhao Liu, Yuyang Huang, Jiayi Yao, Shaoting Feng, Zhuohan Gu, Kuntai Du, Hanchen Li, Yihua Cheng, Junchen Jiang, Shan Lu, Madan Musuvathi, and Esha Choukse. 2025. DroidSpeak: KV Cache Sharing for Cross-LLM Communication and Multi-LLM Serving. *arXiv:2411.02820 [cs.MA]* <https://arxiv.org/abs/2411.02820>
- [93] Miaojia Lu, Xinyu Yan, Shadi Sharif Azadeh, and Pengling Wang. 2024. An adaptive agent-based approach for instant delivery order dispatching: Incorporating task buffering and dynamic batching strategies. *International Journal of Transportation Science and Technology* 13 (2024), 137–154.
- [94] Chenxiang Ma, Zhisheng Ye, Hanyu Zhao, Zehua Yang, Tianhao Fu, Jiaxun Han, Jie Zhang, Yingwei Luo, Xiaolin Wang, Zhenlin Wang, Yong Li, and Diyu Zhou. 2025. Memory Offloading for Large Language Model Inference with Latency SLO Guarantees. *arXiv:2502.08182 [cs.DC]* <https://arxiv.org/abs/2502.08182>
- [95] Jinming Ma, Jiefei Chen, Xiuhong Li, Jiangfei Duan, Haojie Duanmu, Xingcheng Zhang, Chao Yang, and Dahua Lin. 2025. Tropical: Enhancing SLO Attainment in Disaggregated LLM Serving via SLO-Aware Multiplexing. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.
- [96] Chunyang Meng, Haogang Tong, Tianyang Wu, Maolin Pan, and Yang Yu. 2024. Multi-Level ML Based Burst-Aware Autoscaling for SLO Assurance and Cost Efficiency. *arXiv:2402.12962 [cs.SE]* <https://arxiv.org/abs/2402.12962>
- [97] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. 2023. Towards efficient generative large language model serving: A survey from algorithms to systems. *arXiv preprint arXiv:2312.15234 (2023)*.
- [98] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2024. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (La Jolla, CA, USA) (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 932–949. doi:10.1145/3620666.3651335
- [99] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. 2024. Spotserve: Serving generative large language models on preemptible instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1112–1127.

- [100] Moonmoon Mohanty, Gautham Bolar, Preetam Patil, UmaMaheswari Devi, Felix George, Pratibha Moogi, and Parimal Parag. 2025. Deferred prefill for throughput maximization in LLM inference. In *Proceedings of the 5th Workshop on Machine Learning and Systems* (World Trade Center, Rotterdam, Netherlands) (*EuroMLSys '25*). Association for Computing Machinery, New York, NY, USA, 100–106. doi:10.1145/3721146.3721962
- [101] Ilyas Moutawwakil and Régis Pierrard. 2023. Optimum-Benchmark: A framework for benchmarking the performance of Transformers models with different hardwares, backends and optimizations. <https://github.com/huggingface/optimum-benchmark>. Software.
- [102] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prithvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient large-scale language model training on GPU clusters using megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (St. Louis, Missouri) (*SC '21*). Association for Computing Machinery, New York, NY, USA, Article 58, 15 pages. doi:10.1145/3458817.3476209
- [103] Kelvin K. W. Ng, Henri Maxime Demoulin, and Vincent Liu. 2023. Paella: Low-latency Model Serving with Software-defined GPU Scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (*SOSP '23*). Association for Computing Machinery, New York, NY, USA, 595–610. doi:10.1145/3600006.3613163
- [104] Chengyi Nie, Rodrigo Fonseca, and Zhenhua Liu. 2024. Aladdin: Joint Placement and Scaling for SLO-Aware LLM Serving. arXiv:2405.06856 [cs.DC] <https://arxiv.org/abs/2405.06856>
- [105] NVIDIA. 2023. Triton Inference Server. <https://github.com/triton-inference-server/server>.
- [106] Hyungjun Oh, Kihong Kim, Jaemin Kim, Sungkyun Kim, Junyeol Lee, Du-seong Chang, and Jiwon Seo. 2024. ExeGPT: Constraint-Aware Resource Scheduling for LLM Inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (*ASPLOS '24*). New York, NY, USA. doi:10.1145/3620665.3640383
- [107] OpenAI. 2025. GPT-4.5 System Card. <https://openai.com/research/gpt-4-5>. Accessed: 2025-04-30.
- [108] James Pan and Guoliang Li. 2025. A Survey of LLM Inference Systems. arXiv:2506.21901 [cs.DB] <https://arxiv.org/abs/2506.21901>
- [109] Rui Pan, Zhuang Wang, Zhen Jia, Can Karakus, Luca Zancato, Tri Dao, Yida Wang, and Ravi Netravali. 2025. Marconi: Prefix Caching for the Era of Hybrid LLMs. In *Eighth Conference on Machine Learning and Systems*. <https://openreview.net/forum?id=RUaMUu7vMX>
- [110] Jongsoo Park, Maxim Naumov, Protonu Basu, Summer Deng, Aravind Kalaiah, Daya Khudia, James Law, Parth Malani, Andrey Malevich, Satish Nadathur, et al. 2018. Deep learning inference in facebook data centers: Characterization, performance optimizations and hardware implications. *arXiv preprint arXiv:1811.09886* (2018).
- [111] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. doi:10.1109/ISCA59077.2024.00019
- [112] Archit Patke, Dharmath Reddy, Saurabh Jha, Haoran Qiu, Christian Pinto, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2024. Queue Management for SLO-Oriented Large Language Model Serving. In *Proceedings of the 2024 ACM Symposium on Cloud Computing* (Redmond, WA, USA) (*SoCC '24*). Association for Computing Machinery, New York, NY, USA, 18–35. doi:10.1145/3698038.3698523
- [113] Qiangyu Pei, Lin Wang, Dong Zhang, Bingheng Yan, Chen Yu, and Fangming Liu. 2024. InferCool: Enhancing AI Inference Cooling through Transparent, Non-Intrusive Task Reassignment. In *Proceedings of the 2024 ACM Symposium on Cloud Computing* (Redmond, WA, USA) (*SoCC '24*). Association for Computing Machinery, New York, NY, USA, 487–504. doi:10.1145/3698038.3698556
- [114] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, 155–170. <https://www.usenix.org/conference/fast25/presentation/qin>
- [115] Haoran Qiu, Anish Biswas, Zihan Zhao, Jayashree Mohan, Alind Khare, Esha Choukse, Íñigo Goiri, Zeyu Zhang, Haiying Shen, Chetan Bansal, et al. 2025. Modserve: Scalable and resource-efficient large multimodal model serving. *arXiv preprint arXiv:2502.00937* (2025).
- [116] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving Language Understanding by Generative Pre-Training. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf. OpenAI Technical Report.
- [117] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Virtual Event, CA, USA) (*KDD '20*). Association for Computing Machinery, New York, NY, USA, 3505–3506. doi:10.1145/3394486.3406703
- [118] Chaoyi Ruan, Yinhe Chen, Dongqi Tian, Yandong Shi, Yongji Wu, Jialin Li, and Cheng Li. 2025. DynaServe: Unified and Elastic Execution for Dynamic Disaggregated LLM Serving. arXiv:2504.09285 [cs.DC] <https://arxiv.org/abs/2504.09285>
- [119] Souhaila Serbout, Amine El Malki, Cesare Pautasso, and Uwe Zdun. 2023. API Rate Limit Adoption—A pattern collection. In *Proceedings of the 28th European Conference on Pattern Languages of Programs*. 1–20.
- [120] Shivanshu Shekhar, Tanishq Dubey, Koyel Mukherjee, Apoorv Saxena, Atharv Tyagi, and Nishanth Kotla. 2024. Towards Optimizing the Costs of LLM Usage. arXiv:2402.01742 [cs.CL] <https://arxiv.org/abs/2402.01742>
- [121] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in Serving Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 965–988. <https://www.usenix.org/conference/osdi24/presentation/sheng>
- [122] Gursimran Singh, Xinglu Wang, Yifan Hu, Timothy Yu, Linzi Xing, Wei Jiang, Zhefeng Wang, Xiaolong Bai, Yi Li, Ying Xiong, et al. 2024. Efficiently Serving Large Multimodal Models Using EPD Disaggregation. *arXiv preprint arXiv:2501.05460* (2024).

- [123] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 590–606. doi:10.1145/3694715.3695964
- [124] Jovan Stojkovic, Chaojie Zhang, Ñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1348–1362.
- [125] Jovan Stojkovic, Chaojie Zhang, Ñigo Goiri, Esha Choukse, Haoran Qiu, Rodrigo Fonseca, Josep Torrellas, and Ricardo Bianchini. 2025. TAPAS: Thermal- and Power-Aware Scheduling for LLM Inference in Cloud Platforms. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 1266–1281. doi:10.1145/3676641.3716025
- [126] Foteini Strati, Sara Mcallister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. 2024. DéjàVu: KV-cache Streaming for Fast, Fault-tolerant Generative LLM Serving. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 46745–46771. <https://proceedings.mlr.press/v235/strati24a.html>
- [127] Yifan Sui, Hanfei Yu, Yitao Hu, Jianxun Li, and Hao Wang. 2024. Pre-Warming is Not Enough: Accelerating Serverless Inference With Opportunistic Pre-Loading. In *Proceedings of the 2024 ACM Symposium on Cloud Computing* (Redmond, WA, USA) (SoCC '24). Association for Computing Machinery, New York, NY, USA, 178–195. doi:10.1145/3698038.3698509
- [128] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Lumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 173–191.
- [129] Hanshi Sun, Zhuoming Chen, Xinyu Yang, Yuandong Tian, and Beidi Chen. 2024. TriForce: Lossless Acceleration of Long Sequence Generation with Hierarchical Speculative Decoding. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=HVK6nl3i97>
- [130] Ting Sun, Penghan Wang, and Fan Lai. 2025. HyGen: Efficient LLM Serving via Elastic Online-Offline Request Co-location. arXiv:2501.14808 [cs.DC] <https://arxiv.org/abs/2501.14808>
- [131] Yifan Tan, Cheng Tan, Zeyu Mi, and Haibo Chen. 2025. PipeLLM: Fast and Confidential Large Language Model Services with Speculative Pipelined Encryption. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 843–857. doi:10.1145/3669940.3707224
- [132] Yinghao Tang, Tingfeng Lan, Xiuqi Huang, Hui Lu, and Wei Chen. 2025. SCORPIO: Serving the Right Requests at the Right Time for Heterogeneous SLOs in LLM Inference. arXiv:2505.23022 [cs.LG] <https://arxiv.org/abs/2505.23022>
- [133] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).
- [134] PyTorch Team. 2023. TorchServe. <https://github.com/pytorch/serve>.
- [135] The AIBrix Team, Jiaxin Shan, Varun Gupta, Le Xu, Haiyang Shi, Jingyuan Zhang, Ning Wang, Linhui Xu, Rong Kang, Tongping Liu, Yifei Zhang, Yiqing Zhu, Shuowei Jin, Gangmuk Lim, Binbin Chen, Zuzhi Chen, Xiao Liu, Xin Chen, Kante Yin, Chak-Pong Chung, Chenyu Jiang, Yicheng Lu, Jianjun Chen, Caixue Lin, Wu Xiang, Rui Shi, and Liguang Xie. 2025. AIBrix: Towards Scalable, Cost-Effective Large Language Model Inference Infrastructure. arXiv:2504.03648 [cs.DC] <https://arxiv.org/abs/2504.03648>
- [136] Olivier Terzo, Pietro Ruiu, Enrico Bucci, and Fatos Xhafa. 2013. Data as a Service (DaaS) for Sharing and Processing of Large Data Collections in the Cloud. In *2013 Seventh International Conference on Complex, Intelligent, and Software Intensive Systems*. 475–480. doi:10.1109/CISIS.2013.87
- [137] The Celery Project. 2025. *Celery: Distributed Task Queue*. The Celery Project. <https://docs.celeryq.dev/en/stable/>
- [138] Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. 2022. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239* (2022).
- [139] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [140] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [141] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [142] Ashwin K Vijayakumar, Michael Cogswell, Ramprasad R Selvaraju, Qing Sun, Stefan Lee, David Crandall, and Dhruv Batra. 2016. Diverse beam search: Decoding diverse solutions from neural sequence models. *arXiv preprint arXiv:1610.02424* (2016).
- [143] Li Wang, Yankai Jiang, and Ningfang Mi. 2024. Advancing Serverless Computing for Scalable AI Model Inference: Challenges and Opportunities. In *Proceedings of the 10th International Workshop on Serverless Computing*. 1–6.
- [144] Yuxin Wang, Yuhang Chen, Zeyu Li, Xueze Kang, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2024. BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving Systems. arXiv:2401.17644
- [145] Zhican Wang, Hongxiang Fan, Haroon Waris, Gang Wang, Zhenyu Li, Jianfei Jiang, Yanan Sun, and Guanghui He. 2025. VEDA: Efficient LLM Generation Through Voting-based KV Cache Eviction and Dataflow-flexible Accelerator. arXiv:2507.00797 [cs.AR] <https://arxiv.org/abs/2507.00797>

- [146] Zhibin Wang, Shipeng Li, Xue Li, Yuhang Zhou, Zhonghui Zhang, Zibo Wang, Rong Gu, Chen Tian, Kun Yang, and Sheng Zhong. 2025. Echo: Efficient Co-Scheduling of Hybrid Online-Offline Tasks for Large Language Model Serving. *arXiv:2504.03651 [cs.DC]* <https://arxiv.org/abs/2504.03651>
- [147] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. 2022. MLaaS in the Wild: Workload Analysis and Scheduling in Large-Scale Heterogeneous GPU Clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 945–960. <https://www.usenix.org/conference/nsdi22/presentation/weng>
- [148] Grant Wilkins, Srinivasan Keshav, and Richard Mortier. 2025. Offline Energy-Optimal LLM Serving: Workload-Based Energy Models for LLM Inference on Heterogeneous Systems. *SIGENERGY Energy Inform. Rev.* 4, 5 (April 2025), 113–119. doi:10.1145/3727200.3727217
- [149] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 640–654. doi:10.1145/3694715.3695948
- [150] Bingyang Wu, Ruidong Zhu, Zili Zhang, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. {dLoRA}: Dynamically orchestrating requests and adapters for {LoRA}{LLM} serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 911–927.
- [151] Yu Wu, Tongxuan Liu, Yuting Zeng, Siyu Wu, Jun Xiong, Xianzhe Dong, Hailong Yang, Ke Zhang, and Jing Li. 2025. Arrow: Adaptive Scheduling Mechanisms for Disaggregated LLM Inference Architecture. *arXiv:2505.11916 [cs.DC]* <https://arxiv.org/abs/2505.11916>
- [152] Tian Xia, Ziming Mao, Jamison Kerney, Ethan J. Jackson, Zhifei Li, Jiarong Xing, Scott Shenker, and Ion Stoica. 2025. SkyLB: A Locality-Aware Cross-Region Load Balancer for LLM Inference. *arXiv:2505.24095 [cs.DC]* <https://arxiv.org/abs/2505.24095>
- [153] Yuxing Xiang, Xue Li, Kun Qian, Wenyan Yu, Ennan Zhai, and Xin Jin. 2025. ServeGen: Workload Characterization and Generation of Large Language Model Serving in Production. *arXiv:2505.09999 [cs.DC]* <https://arxiv.org/abs/2505.09999>
- [154] Jiale Xu, Rui Zhang, Yi Xiong, Cong Guo, Zihan Liu, Yangjie Zhou, Weiming Hu, Hao Wu, Changxu Shao, Ziqing Wang, Yongjie Yuan, Junping Zhao, Minyi Guo, and Jingwen Leng. 2025. eLLM: Elastic Memory Management Framework for Efficient LLM Serving. *arXiv:2506.15155 [cs.DC]* <https://arxiv.org/abs/2506.15155>
- [155] Xusheng Xu, Jiangyu Cui, Zidong Cui, Runhong He, Qingyu Li, Xiaowei Li, Yanling Lin, Jiale Liu, Wuxin Liu, Jiale Lu, et al. 2024. MindSpore Quantum: a user-friendly, high-performance, and AI-compatible quantum computing framework. *arXiv preprint arXiv:2406.17248* (2024).
- [156] Yang Xu, Lei Wu, Liying Guo, Zheng Chen, Lai Yang, and Zhongzhi Shi. 2011. An Intelligent Load Balancing Algorithm Towards Efficient Cloud Computing.. In *AI for Data Center Management and Cloud Computing*.
- [157] Mo Xuan, Zhang yue, and Wu Weigang. 2025. MaaS: SLO-aware Orchestration of Heterogeneous Model Instances for MaaS. *arXiv:2509.06362 [cs.DC]* <https://arxiv.org/abs/2509.06362>
- [158] Zheming Yang, Yuanhao Yang, Chang Zhao, Qi Guo, Wenkai He, and Wen Ji. 2024. PerLLM: Personalized Inference Scheduling with Edge-Cloud Collaboration for Diverse LLM Services. *arXiv:2405.14636 [cs.DC]* <https://arxiv.org/abs/2405.14636>
- [159] Jiayi Yao, Hanchen Li, Yuhang Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) (EuroSys '25). Association for Computing Machinery, New York, NY, USA, 94–109. doi:10.1145/3689031.3696098
- [160] Zhiwei Yao, Yang Xu, Hongli Xu, Yunming Liao, and Zuan Xie. 2025. Efficient Deployment of Large Language Models on Resource-constrained Devices. *arXiv:2501.02438 [cs.LG]* <https://arxiv.org/abs/2501.02438>
- [161] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. 2023. Zeus: Understanding and optimizing {GPU} energy consumption of {DNN} training. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 119–139.
- [162] Zahra Yousefijamrani, Xinglu Wang, Qian Wang, Morgan Lindsay Heisler, Taha Shabani, Niloofar Gholipour, Parham Yassini, Hong Chang, Kan Chen, Qiantao Zhang, Xiaolong Bai, Jiannan Wang, Ying Xiong, Yong Zhang, and Zhenan Fan. 2025. HyperFlexis: Joint Design of Algorithms and Systems for Multi-SLO Serving and Fast Scaling. *arXiv:2508.15919 [cs.DC]* <https://arxiv.org/abs/2508.15919>
- [163] Chengye Yu, Tianyu Wang, Zili Shao, Linjie Zhu, Xu Zhou, and Song Jiang. 2024. TwinPilots: A New Computing Paradigm for GPU-CPU Parallel LLM Inference. In *Proceedings of the 17th ACM International Systems and Storage Conference* (Virtual, Israel) (SYSTOR '24). Association for Computing Machinery, New York, NY, USA, 91–103. doi:10.1145/3688351.3689164
- [164] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [165] Minchen Yu, Rui Yang, Chaobo Jia, Zhaoyuan Su, Sheng Yao, Tingfeng Lan, Yuchen Yang, Yue Cheng, Wei Wang, Ao Wang, and Ruichuan Chen. 2025. λScale: Enabling Fast Scaling for Serverless Large Language Model Inference. *arXiv:2502.09922 [cs.DC]* <https://arxiv.org/abs/2502.09922>
- [166] Shan Yu, Jiarong Xing, Yifan Qiao, Mingyuan Ma, Yangmin Li, Yang Wang, Shuo Yang, Zhiqiang Xie, Shiyi Cao, Ke Bao, et al. 2025. Prism: Unleashing GPU Sharing for Cost-Efficient Multi-LLM Serving. *arXiv preprint arXiv:2505.04021* (2025).
- [167] Shaoyun Zeng, Minhui Xie, Shiwei Gao, Youmin Chen, and Youyou Lu. 2025. Medusa: Accelerating Serverless LLM Inference with Materialization. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 653–668. doi:10.1145/3669940.3707285
- [168] Chengliang Zhang, Minchen Yu, Wei Wang, and Feng Yan. 2019. {MArk}: Exploiting cloud services for {Cost-Effective}, {SLO-Aware} machine learning inference serving. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. 1049–1062.
- [169] Dingyan Zhang, Haotian Wang, Yang Liu, Xingda Wei, Yizhou Shan, Rong Chen, and Haibo Chen. 2024. Fast and Live Model Auto Scaling with O (1) Host Caching. *arXiv preprint arXiv:2412.17246* (2024).

- [170] Hongbin Zhang, Taosheng Wei, Zhenyi Zheng, Jiangsu Du, Zhiguang Chen, and Yutong Lu. 2025. TD-Pipe: Temporally-Disaggregated Pipeline Parallelism Architecture for High-Throughput LLM Inference. *arXiv:2506.10470 [cs.DC]* <https://arxiv.org/abs/2506.10470>
- [171] Xiang Zhang, Shaojie Shen, et al. 2023. vLLM: Easy, Fast, and Cheap LLM Serving with Open-Source Infrastructure. *arXiv preprint arXiv:2309.06180* (2023).
- [172] Yihao Zhao, Jiadun Chen, Peng Sun, Lei Li, Xuanzhe Liu, and Xin Jin. 2025. SeaLLM: Service-Aware and Latency-Optimized Resource Sharing for Large Language Model Inference. *arXiv:2504.15720 [cs.DC]* <https://arxiv.org/abs/2504.15720>
- [173] Yongli Zhao, Wei Wang, Yajie Li, Carlos Colman Meixner, Massimo Tornatore, and Jie Zhang. 2019. Edge Computing and Networking: A Survey on Infrastructures and Applications. *IEEE Access* 7 (2019), 101213–101230. doi:10.1109/ACCESS.2019.2927538
- [174] Ranran Zhen, Juntao Li, Yixin Ji, Zhenlin Yang, Tong Liu, Qingrong Xia, Xinyu Duan, Zhefeng Wang, Baoxing Huai, and Min Zhang. 2025. Taming the Titans: A Survey of Efficient LLM Inference Serving. *arXiv:2504.19720 [cs.CL]* <https://arxiv.org/abs/2504.19720>
- [175] Siyuan Zheng, Ji Lin, Yuze Zhang, et al. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. *arXiv preprint arXiv:2302.13971* (2023).
- [176] Yue Zheng, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanchao Shu, and Jiming Chen. 2024. A review on edge large language models: Design, execution, and applications. *Comput. Surveys* (2024).
- [177] Zhen Zheng, Xin Ji, Taosong Fang, Fanghao Zhou, Chuanjie Liu, and Gang Peng. 2025. BatchLLM: Optimizing Large Batched LLM Inference with Global Prefix Sharing and Throughput-oriented Token Batching. *arXiv:2412.03594 [cs.CL]* <https://arxiv.org/abs/2412.03594>
- [178] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. 2023. Response Length Perception and Sequence Scheduling: An LLM-Empowered LLM Inference Pipeline. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.). https://proceedings.neurips.cc/paper_files/paper/2023/file/ce7ff3405c782f761fac7f849b41ae9a-Paper-Conference.pdf
- [179] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
- [180] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, et al. 2024. A Survey on Efficient Inference for Large Language Models. *CoRR* (2024).